

FH AACHEN  
UNIVERSITY OF APPLIED SCIENCES

Faculty of Aerospace Engineering

BACHELOR'S THESIS



# **Usage of Gravity Assists and Aerobraking at Titan for a Mission to Enceladus**

Jason Cedric Kalscheuer  
3307475

March 19, 2025

Supervisor:

Prof. Dr.-Ing. Bernd Dachwald

Co-Supervisor:

Fabienne Seibert, M.Sc.

## **Abstract**

Space exploration requires both acceleration and deceleration during a mission. The Saturnian moon Enceladus recently became a high-priority target for further scientific investigations. A mission to Enceladus requires multiple significant changes to the spacecraft's energy. This thesis investigates the feasibility of using aerobraking and gravity assists at the Saturnian moon Titan to minimize fuel requirements for reaching a stable orbit around Enceladus. Using the self-developed Titan Aerobraking and Gravity Assists Simulator (TAGAS), trajectories for various hyperbolic excess velocities were examined and optimized for mission duration and thermal loads. Titan's atmosphere proved highly effective in altering the spacecraft's velocity to establish a stable Enceladus-synchronized orbit without spending large amounts of fuel. The validation of TAGAS was mainly done by comparing the predicted results with a variety of NASA studies and mission data from Cassini-Huygens.

# Contents

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>List of Figures</b>                              | <b>5</b>  |
| <b>List of Tables</b>                               | <b>7</b>  |
| <b>Nomenclature</b>                                 | <b>8</b>  |
| <b>1 Introduction</b>                               | <b>10</b> |
| 1.1 Motivation . . . . .                            | 10        |
| 1.2 Objectives . . . . .                            | 10        |
| <b>2 Background</b>                                 | <b>12</b> |
| 2.1 Orbital Dynamics . . . . .                      | 12        |
| 2.1.1 Basics of Interplanetary Travel . . . . .     | 12        |
| 2.1.2 Gravity Assist and Aerobraking . . . . .      | 15        |
| 2.1.3 History of Aerobraking . . . . .              | 16        |
| 2.1.4 Thermal Protection Systems . . . . .          | 17        |
| 2.2 The Saturnian System . . . . .                  | 18        |
| <b>3 Description of the TAGAS Program Code</b>      | <b>22</b> |
| 3.1 User Input . . . . .                            | 22        |
| 3.2 Program Preparation . . . . .                   | 28        |
| 3.3 Atmospheric Model . . . . .                     | 29        |
| 3.4 Equations of Motion . . . . .                   | 30        |
| 3.5 Launch Delay Calculation . . . . .              | 34        |
| 3.6 Aerobraking Altitude Adjustment . . . . .       | 36        |
| <b>4 Results</b>                                    | <b>40</b> |
| 4.1 Validation of TAGAS . . . . .                   | 40        |
| 4.1.1 Validation of the Atmospheric Model . . . . . | 40        |
| 4.1.2 Validation of Gravitational Forces . . . . .  | 43        |
| 4.1.3 Validation of Aerodynamic Forces . . . . .    | 44        |
| 4.1.4 Validation of Aerothermodynamics . . . . .    | 46        |
| 4.2 Simulation Results . . . . .                    | 48        |
| <b>5 Conclusion</b>                                 | <b>57</b> |
| 5.1 Summary . . . . .                               | 57        |
| 5.2 Outlook . . . . .                               | 57        |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>A</b> | <b>Additional Figures</b>                                  | <b>59</b> |
| A.1      | Orbit of All Saturnian Moons . . . . .                     | 59        |
| A.2      | Full Trajectory Plot . . . . .                             | 60        |
| A.3      | Deviation in Aerothermodynamics . . . . .                  | 61        |
| <b>B</b> | <b>Reference Frames</b>                                    | <b>64</b> |
| B.1      | Inertial Cartesian Reference Frame $\mathcal{I}$ . . . . . | 64        |
| B.2      | Orbit Reference Frame $\mathcal{P}$ . . . . .              | 65        |
| <b>C</b> | <b>Python Code</b>                                         | <b>66</b> |
| C.1      | Launch Delay Iteration Process . . . . .                   | 66        |
| C.2      | Adjust Aerobrake Altitude Process . . . . .                | 68        |
|          | <b>Bibliography</b>                                        | <b>70</b> |



# List of Figures

|      |                                                                     |    |
|------|---------------------------------------------------------------------|----|
| 2.1  | Patched conics approximation (Parvathi et al., 2016)                | 13 |
| 2.2  | Arrival phase visualization (Dachwald, 2024b)                       | 14 |
| 2.3  | Gravity assist energy increase (Brown, 1998)                        | 15 |
| 2.4  | Gravity assist energy decrease (Brown, 1998)                        | 15 |
| 2.5  | Saturn moon size comparison (Seal, 2002)                            | 19 |
| 2.6  | Orbit of Saturn's major moons (Winter, 2021)                        | 20 |
| 2.7  | Atmospheric density comparison (Justus et al., 2003)                | 21 |
| 3.1  | Trajectory plot                                                     | 24 |
| 3.2  | Atmospheric model plot                                              | 24 |
| 3.3  | Data over time plot                                                 | 25 |
| 3.4  | Possible altitudes for orbit capture at Titan                       | 26 |
| 3.5  | Heat loads during aerobraking                                       | 27 |
| 3.6  | Initial spacecraft location for different arrival velocities        | 31 |
| 3.7  | Logic of the launch delay iteration                                 | 35 |
| 3.8  | Delay iteration example                                             | 35 |
| 3.9  | Final print statement                                               | 37 |
| 3.10 | Two vs. three flyby comparison                                      | 38 |
| 3.11 | Logic of the aerobrake altitude iteration                           | 39 |
| 4.1  | Atmospheric data from HASI (Jet Propulsion Laboratory, 2019)        | 41 |
| 4.2  | Comparison of TAGAS atmospheric model and HASI data                 | 42 |
| 4.3  | Titan orbiter transfer trajectory (Nixon et al., 2016)              | 43 |
| 4.4  | Titan orbiter arrival trajectory (Nixon et al., 2016)               | 43 |
| 4.5  | Spacecraft velocity when reaching Titan                             | 44 |
| 4.6  | HYPAS aerocapture guide (Way et al., 2003)                          | 45 |
| 4.7  | TAGAS aerocapture guide                                             | 46 |
| 4.8  | Heatflux during aerocapture according to NASA study (Laub, 2012)    | 47 |
| 4.9  | Heatflux during aerocapture according to TAGAS                      | 48 |
| 4.10 | Comparison of apoapsis based on flyby altitude                      | 49 |
| 4.11 | Lowest possible aerobraking altitude                                | 50 |
| 4.12 | Thermal load at minimum altitude (incl. safety margin)              | 51 |
| 4.13 | Possible flyby altitudes for orbit capture with altitude boundaries | 52 |
| 4.14 | Orbit for $v_{inf} = 5.5$ km/s                                      | 53 |
| 4.15 | Thermal loads for $v_{inf} = 5.5$ km/s                              | 53 |
| 4.16 | Final trajectory                                                    | 54 |
| 4.17 | Thermal loads of the final trajectory                               | 55 |

|      |                                                                                                           |    |
|------|-----------------------------------------------------------------------------------------------------------|----|
| 4.18 | Orbital period of the final trajectory . . . . .                                                          | 56 |
| A.1  | Saturn and all known moons (Barnett, 2025) . . . . .                                                      | 59 |
| A.2  | Full Trajectory Plot . . . . .                                                                            | 60 |
| A.3  | Atmosphere model according to NASA aerothermodynamics study (Laub, 2012) . . . . .                        | 61 |
| A.4  | Atmosphere model from TAGAS . . . . .                                                                     | 62 |
| A.5  | Altitude & velocity during aerocapture according to NASA aerothermodynamics study (Laub, 2012)) . . . . . | 62 |
| A.6  | Velocity during aerocapture from TAGAS . . . . .                                                          | 63 |
| A.7  | Altitude during aerocapture from TAGAS . . . . .                                                          | 63 |
| B.1  | Cartesian reference frame (Dachwald, 2024a) . . . . .                                                     | 64 |
| B.2  | Orbit reference frame (Dachwald, 2023) . . . . .                                                          | 65 |

# List of Tables

|     |                                                                           |    |
|-----|---------------------------------------------------------------------------|----|
| 2.1 | Heat sink material properties (Haynes, 2017) . . . . .                    | 17 |
| 2.2 | Titan and Enceladus Constants (Jet Propulsion Laboratory, 2022) . . . .   | 20 |
| 3.1 | Rocket parameters . . . . .                                               | 22 |
| 3.2 | Output parameters . . . . .                                               | 23 |
| 3.3 | Planetary and orbital constants (Jet Propulsion Laboratory, 2022) . . . . | 29 |
| 3.4 | Spacecraft status parameters . . . . .                                    | 32 |
| 3.5 | Equations of motion formula symbols . . . . .                             | 33 |
| 3.6 | Database entry . . . . .                                                  | 36 |

# Nomenclature

## Greek Letters

|          |                                 |                         |
|----------|---------------------------------|-------------------------|
| $\alpha$ | rad                             | Flight path angle       |
| $\beta$  | rad                             | Thrust angle            |
| $\eta$   | –                               | Throttle parameter      |
| $\mu$    | km <sup>3</sup> /s <sup>2</sup> | Gravitational parameter |
| $\phi$   | rad                             | Angular position        |
| $\rho$   | kg/m <sup>3</sup>               | Density                 |

## Latin Letters

|                    |                   |                            |
|--------------------|-------------------|----------------------------|
| $\hat{\mathbf{r}}$ | –                 | Directional unit vector    |
| $a$                | km                | Semi-major axis            |
| $A$                | m <sup>2</sup>    | Cross-sectional Area       |
| $c_d$              | –                 | Drag coefficient           |
| $c_{eff}$          | km/s              | Effective exhaust velocity |
| $c$                | kJ/(kg K)         | Specific heat capacity     |
| $D$                | kN                | Force of airdrag           |
| $e$                | –                 | Eccentricity               |
| $E$                | MJ/kg             | Specific orbital Energy    |
| $F$                | N                 | Force                      |
| $g_0$              | m/s               | Gravitational acceleration |
| $h$                | km/s              | Specific angular momentum  |
| $H$                | MJ/kg             | Enthalpy                   |
| $I_{sp}$           | s                 | Specific impulse           |
| $M$                | kg                | Planet mass                |
| $m$                | kg                | Mass                       |
| $n$                | 1/s               | Mean motion                |
| $Q$                | J/cm <sup>2</sup> | Heat load                  |
| $q$                | W/cm <sup>2</sup> | Heat rate                  |
| $R_n$              | m                 | Aeroshell radius           |

|     |      |                 |
|-----|------|-----------------|
| $r$ | km   | Radial distance |
| $r$ | km   | Radius          |
| $t$ | cm   | Thickness       |
| $T$ | K    | Temperature     |
| $v$ | km/s | Velocity        |

### Acronyms & Abbreviations

|       |                                                     |
|-------|-----------------------------------------------------|
| EPS   | Electrical propulsion system                        |
| HEEET | Heatshield for Extreme Entry Environment Technology |
| SOI   | Sphere of influence                                 |
| TAGAS | Titan Aerobraking and Gravity Assist Simulator      |
| TPS   | Thermal protection system                           |

# 1 Introduction

## 1.1 Motivation

Enceladus is a small, icy moon in the Saturnian system that has gained considerable attention over the past few years due to numerous scientific discoveries. The most important is that Enceladus meets the three requirements for a habitable environment: the presence of liquid water, a source of energy, and a specific set of chemical elements (ESA, 2024). These discoveries make Enceladus an optimal target for space exploration missions. Yet, no space agency has managed to land on Enceladus to date. One of the reasons for that is the massive amount of deceleration required in the Saturnian system to establish orbit around it. Traditional deceleration methods relying on thrusters require large amounts of fuel, thus limiting the mass available for scientific payloads. This thesis investigates an alternative approach. Gravity assists and aerobraking maneuvers have been used in previous missions at Venus and Mars and have proven to be effective and reliable ways of changing a spacecraft's orbit without fuel (ESA, 2017). However, Enceladus' small mass and non-existent atmosphere are challenges that need to be overcome. Leveraging another celestial body's gravity and atmosphere for aerobraking might reduce fuel requirements and allow for better mission objectives in future missions toward Enceladus.

## 1.2 Objectives

This thesis will elaborate on a computer program used to simulate different trajectories around Saturn, including its moons Titan and Enceladus. The Titan Aerobraking and Gravity Assist Simulator (TAGAS). The program's results will be analyzed to assess the possibilities of establishing an orbit around Enceladus utilizing aerobraking and gravity assists and the effects of such maneuvers on the mission duration and orbiter structure due to generated heat loads. The problem to solve in this thesis is finding multiple trajectories for various arrival velocities. The spacecraft's starting point is located at the

edge of Saturn's sphere of influence. The exact starting location depends on the arrival velocity respective to Saturn. It is set so that the crossing point with Titan's orbit on its first flyby is identical, ensuring comparable results. The final orbit to be attained has its periapsis within the sphere of influence of Enceladus and the apoapsis close to Titan. The exact apoapsis height will vary depending on the velocity since the orbital period should be synchronized with Enceladus, ensuring that each time the spacecraft reaches periapsis, Enceladus is also present for potential rendezvous or orbit insertion opportunities. Several variables are optimized, such as time of flight, thermal loads generated during aerobraking, and the resulting mass gain of the heat protection systems. Although a lower arrival velocity seems logical to reduce deceleration requirements, Saturn's significant gravitational influence gives a lower boundary on the slowest arrival velocity feasible. Arriving with a low velocity in the sphere of influence risks a trajectory that gets pulled into Saturn and ends in mission loss. Hence, a balance between reducing approach speed and maintaining a viable capture trajectory is required. According to this, the arrival velocities examined are between 0.5 km/s and 4.25 km/s.

## 2 Background

### 2.1 Orbital Dynamics

#### 2.1.1 Basics of Interplanetary Travel

Interplanetary travel consists of at least four bodies (Dachwald, 2024b):

1. Departure body - in this case Earth
2. Arrival body - in this case Enceladus
3. The Sun
4. The spacecraft

Calculating the forces of all bodies onto each other is called an N-body simulation. Using it for the four bodies listed above is almost impossible due to technical limitations and the respective influence on each other's trajectory. Hence, mission planning typically employs the patched conics approximation. This method divides the mission into three distinct phases: departure, transfer, and arrival. Each phase only consists of the spacecraft and the dominant gravitational source during that phase. Therefore, the problem gets split into multiple two-body problems, which are significantly easier to solve. The transition between these phases is determined by the concept of the sphere of influence (SOI). A sphere of influence is the region around a celestial body where its gravitational pull exceeds others, making it the dominant force (Dachwald, 2024b). The radius of the sphere of influence is given by:

$$r_{SOI} = a \left( \frac{\mu_{body}}{\mu_{Sun}} \right)^{\frac{2}{5}} \quad (2.1)$$

where  $a$  is the semi-major axis of the celestial body's orbit,  $\mu_{body}$  is the gravitational parameter of the body, for example, Earth, and  $\mu_{Sun}$  is the Sun's gravitational parameter (Bate et al., 1971). Figure 2.1 illustrates the three-phase approximation applied to an interplanetary journey between Earth and Mars.



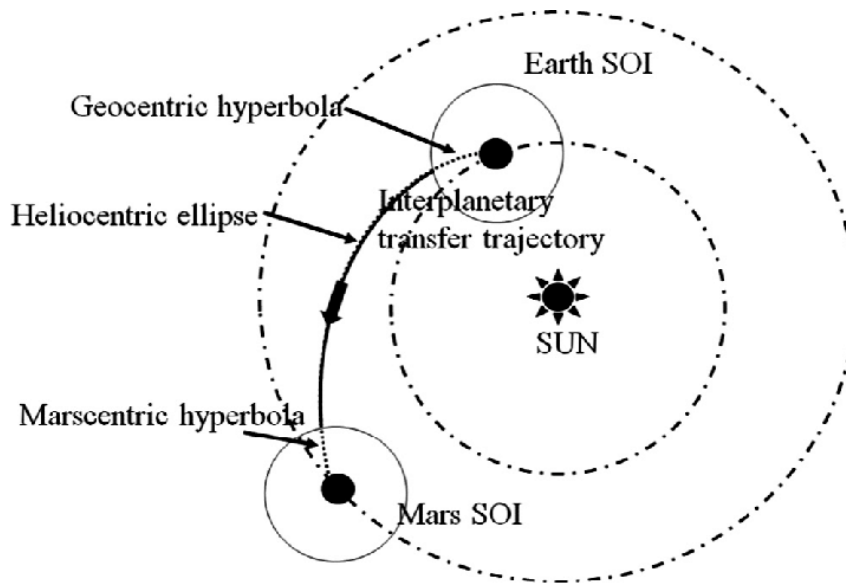


Figure 2.1: Patched conics approximation (Parvathi et al., 2016)

For the examined mission in this thesis, there are two more bodies: Saturn and Titan. These extra bodies make an N-body simulation even less feasible. The patched conics approximation divides the mission and makes it possible to focus only on the arrival phase while receiving valuable results and insights. In interplanetary travel, the spacecraft's velocity is one of the most important parameters to track and adjust. A spacecraft arriving in the Saturnian system has excessive arrival velocities, requiring deceleration to capture it into a stable orbit (Wolf et al., 1995). Figure 2.2 shows the arrival of a spacecraft at Mars. The flyby trajectory marked in green is the path a spacecraft would take without any deceleration using thrust or aerobraking. In contrast, the capture orbit marked in blue is the target trajectory of the spacecraft.

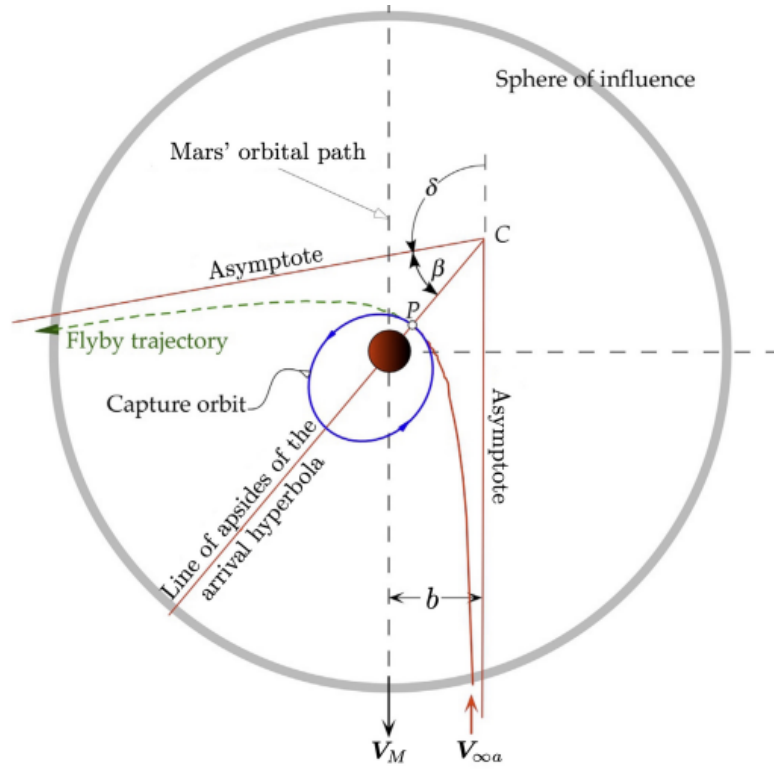


Figure 2.2: Arrival phase visualization (Dachwald, 2024b)

The spacecraft needs to be decelerated at point P, called periapsis, to achieve the blue capture orbit instead of the green flyby trajectory. It is the point on the trajectory where the spacecraft is closest to the central body of an orbit (Curtis, 2014). The velocity is fundamentally defined by the vis-viva equation:

$$v = \sqrt{\frac{2\mu}{r} - \frac{\mu}{a}} \quad (2.2)$$

Since the gravitational parameter  $\mu$  and the distance of the spacecraft to the center of mass  $r$  have not changed, the semi-major axis ( $a$ ) must change. The semi-major axis is defined as:

$$a = \frac{r_p + r_a}{2} \quad (2.3)$$

Since the spacecraft is located at the periapsis  $r_p$ , the only parameter that can change is the orbit's farthest point, the apoapsis  $r_a$  (Curtis, 2014).

### 2.1.2 Gravity Assist and Aerobraking

A gravity assist describes a spacecraft maneuver using the gravitational force of a celestial body for its benefit. It is mainly used to accelerate or decelerate a spacecraft relative to the central body on its trajectory and significantly alters its trajectory after the maneuver. Flying ahead or in front of a celestial body on its orbit, the spacecraft experiences a gravitational force of said body that acts in the opposing direction of the spacecraft's velocity vector. This results in the spacecraft losing energy and reducing its velocity relative to the central body during the maneuver. The same principle works for flying behind a celestial body on its orbit, increasing the spacecraft's velocity. The Cassini spacecraft, for example, used a Venus flyby to change its velocity relative to the Sun by km/s (Jet Propulsion Laboratory, 2019). Figure 2.3 and Figure 2.4 show both cases.

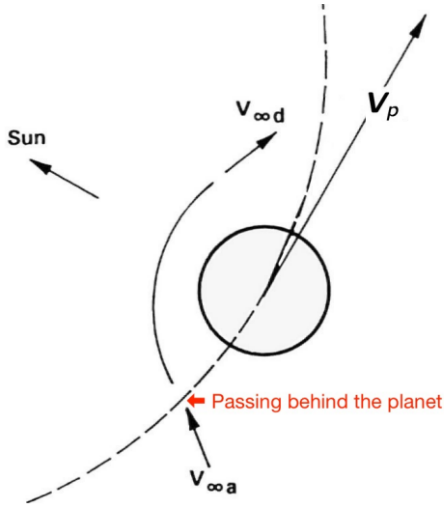


Figure 2.3: Gravity assist energy increase (Brown, 1998)

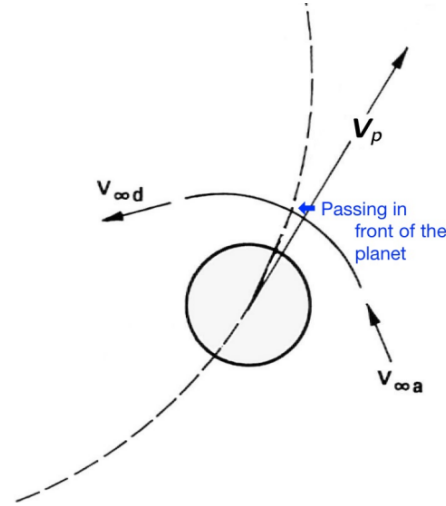


Figure 2.4: Gravity assist energy decrease (Brown, 1998)

The vector of force  $\mathbf{F}$  acting onto the spacecraft can be described by Newton's law of universal gravitation (Roy, 2005). Using  $M$  as the mass of the planet,  $m$  as the spacecraft's mass and  $\hat{\mathbf{r}}$  as the unit vector giving the direction, the force is described by the following equation:

$$\mathbf{F} = -\mathcal{G} \frac{Mm}{r^2} (-\hat{\mathbf{r}}) \quad (2.4)$$

Since the gravitational force that one body exerts on another decreases with the square of the distance between them, the change in velocity depends on the distance of the flyby

maneuver. It is imperative to note that Newton's laws only apply in an inertial frame. An inertial frame is neither rotated nor accelerated. More information on reference frames is located in Appendix B. An even more advanced velocity reduction is possible on a celestial body with an atmosphere. Aerobraking describes a spacecraft maneuver using the atmospheric drag experienced when flying through an atmosphere at certain altitudes to reduce its velocity.  $c_d$  is a unit less coefficient that describes how aerodynamic a body is. Together with the density of the atmosphere  $\rho$ , the cross-sectional area  $A$ , and the velocity  $v$ , the force of the air drag can be calculated with (Hall, 2023):

$$D = \frac{1}{2}c_d\rho Av^2 \quad (2.5)$$

Because this force acts opposite to the direction of travel, it further decreases the spacecraft's energy. This maneuver is not feasible on all celestial bodies. For a celestial body with a very low atmospheric density, the impact on trajectory may not be strong enough, or the required altitude to the body is too low to keep safety margins. Since aerobraking maneuvers are significantly closer to the celestial body than gravity assists, errors can potentially result in a loss of mission. Hence, aerobraking requires significantly more precise trajectory design and planning. When done correctly, however, aerobraking can have a tremendous positive impact on the mission.

### 2.1.3 History of Aerobraking

In the past 30 years, aerobraking and gravity assists have become reliable ways of adjusting a spacecraft's trajectory without spending fuel. It has become a central strategy in modern mission planning to minimize fuel costs and maximize payload. The first test to see if aerobraking was feasible happened in 1991. The Hiten spacecraft from ISAS from Japan used Earth's atmosphere to slow down by 1.71 m/s (Ahrens, 2021). Since then, it has been successfully used in various missions around Venus and Mars and has not only reduced the fuel requirements of those missions but has also enabled missions that were unattainable without the velocity reduction provided by the aerobraking maneuver like the Magellan mission on Venus in 1993 (Jet Propulsion Laboratory, 1993). In 2014, the European Space Agency also used aerobraking for the first time with the Venus Express (ESA, 2014). After the successful test, the ESA used aerobraking to lower the

orbit of the ExoMars Trace Gas Orbiter mission from 33,000 km to 400 km, mainly using the atmosphere of Mars (ESA, 2017).

## 2.1.4 Thermal Protection Systems

There are three ways to protect a spacecraft from thermal loads:

1. Radiative thermal solution via coatings offers constant heat dissipation with no additional weight. It is the only thermal protection system where the time spent in the atmosphere does not matter. This characteristic makes it the optimal thermal solution if the heat loads allow it. Modern coatings, designed for in-atmospheric use, can keep an emissivity of 0.9 up to 1773 K, resulting in up to 50.4267 W/cm<sup>2</sup> (Shao et al., 2019).

$$q = 0.9 \times 5.67 \times 10^{-12} \text{ W}/(\text{cm}^2 \text{ K}^4) \times (1773 \text{ K})^4 = 50.4267 \text{ W}/\text{cm}^2 \quad (2.6)$$

2. Heat sink solutions store the heat during atmospheric phases and dissipate it during the rest of the orbit. Aluminum and copper are common materials used for heat sinks. Aluminum, however, offers a better ratio of thermal conductivity when put into perspective of its density and, therefore, the weight of the heat sink. The material properties from Table 2.1 and Equation 2.7 deliver the following results for heat sinks:

| Property                   | Aluminum | Copper | Unit              | Formula symbol |
|----------------------------|----------|--------|-------------------|----------------|
| Specific heat capacity     | 0.897    | 0.385  | kJ/(kg K)         | $c$            |
| Material density           | 2700     | 8960   | kg/m <sup>3</sup> | $\rho$         |
| Allowed temperature change | 100      | 100    | K                 | $K$            |
| Material thickness         | 10       | 10     | cm                | $t$            |
| Possible heat load         | 2421.9   | 3434.2 | J/cm              | $Q$            |

Table 2.1: Heat sink material properties (Haynes, 2017)

$$Q = c \rho t \Delta T \quad (2.7)$$

3. Ablative thermal protection systems heat up until they vaporize to dissipate the heat. The current state-of-the-art material used for this application is Phenolic Impregnated Carbon Ablator, also known as PICA and offers a specific heat capacity  $c_p(T)$  between 983.9 J/(kg K) at 298 K up to 2001 J/(kg K) at 2222 K and an enthalpy of pyrolysis  $\Delta H_{pyrolysis}$  of 29.5 MJ/kg (Wang et al., 2019). The total heat required to vaporize PICA results in 69.4 MJ/kg. Using a density  $\rho$  of 266 kg/m<sup>3</sup>, an ablative thermal protection system made from a 100 mm thick  $t$  PICA material can handle up to 1846 MJ/m<sup>2</sup>.

$$Q_{total} = \left( \int_{T_0}^{T_{vaporization}} c_p(T) dT + \Delta H_{pyrolysis} \right) \rho t \quad (2.8)$$

The next generation of ablative thermal protection will be the Heatshield for Extreme Entry Environment Technology (HEEET), which is still in development. NASA plans on using HEEET for future missions to Saturn, so it is likely that it will be used in the first missions toward Enceladus and will be able to withstand even higher heat loads (Gasch et al., 2022).

## 2.2 The Saturnian System

The Saturnian system consists of Saturn along with its currently known 146 moons. The last discovery of new moons in the Saturnian system was in 2020, making it likely that other moons are yet to be discovered. This makes it the planet with the most moons known to humanity (Barnett, 2025). A full view of Saturn and all known moons is in the appendix at Figure A.1. The moons range from Aegaeon, with a mean radius of 0.12 km, up to Titan, with an equatorial radius of 2575.5 km, is larger than Mercury (Lindsey, 2024). Figure 2.5 visualizes the size of the moons.

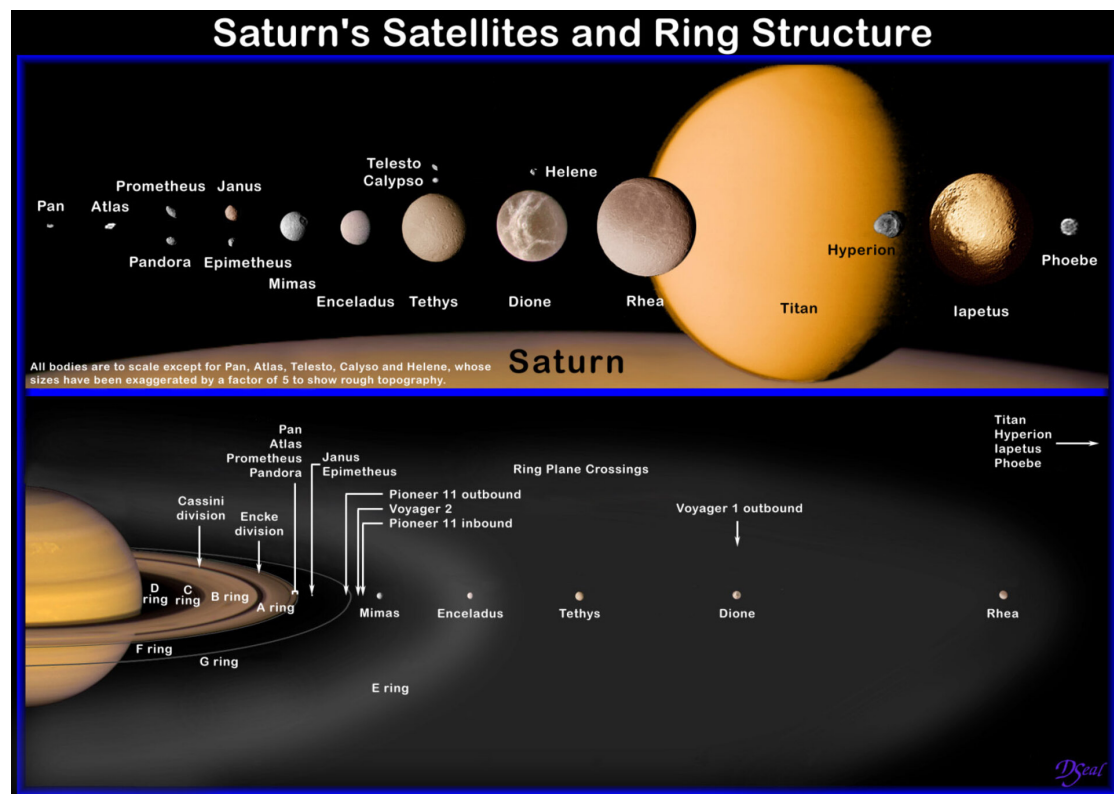


Figure 2.5: Saturn moon size comparison (Seal, 2002)

Of the seven major moons, Titan is the second furthest away, only Iapetus being more than twice as distant to Saturn. Enceladus, in comparison, is the second closest to Saturn. Figure 2.6 delivers an overview of the orbits of Saturn's major moons.

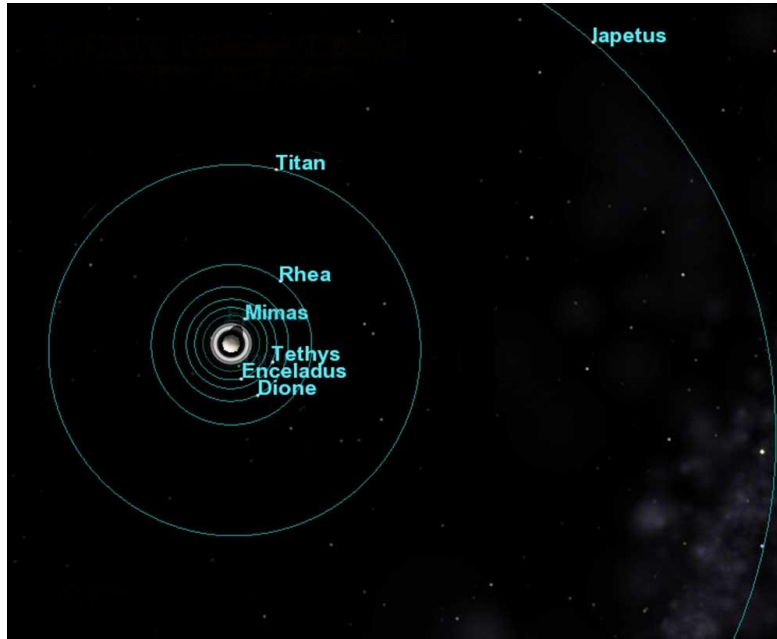


Figure 2.6: Orbit of Saturn's major moons (Winter, 2021)

Although the target planet of this thesis is Enceladus, Titan does have a significant role in reaching it. Enceladus' small mass results in a sphere of influence radius of about only 488 km, making both aerobraking and gravity assist not a viable option. The Saturnian system, however, does offer a different moon for aerobraking. Titan's atmosphere has been characterized by the Cassini-Huygens mission in 2005. Titan's atmospheric density profile and mass theoretically allow for various aerobraking and gravity assist maneuvers around it (Jet Propulsion Laboratory, 2019). Table 2.2 compares planetary constants of Enceladus and Titan.

| Constant       | Titan                    | Enceladus               | Unit |
|----------------|--------------------------|-------------------------|------|
| Mass           | $13455.3 \times 10^{19}$ | $10.805 \times 10^{19}$ | kg   |
| Radius         | 2575.5                   | 252.3                   | km   |
| Orbital period | 15.945421                | 1.370218                | d    |
| Periapsis      | 1,186,680                | 236,921                 | km   |
| Apoapsis       | 1,257,060                | 239,159                 | km   |

Table 2.2: Titan and Enceladus Constants (Jet Propulsion Laboratory, 2022)



Not only is Titan the only known moon to have a substantial atmosphere, but its atmosphere is also much denser at higher altitudes than that of other planets. Figure 2.7 shows that the required altitude above Titan for aerobraking is 700 km, while the same density is found at altitudes of 100 km or lower on Mars. This higher density gives aerobraking or aerocapture missions at Titan a larger safety margin for perturbations not predicted by a simulation and decreases the risk of mission failure.

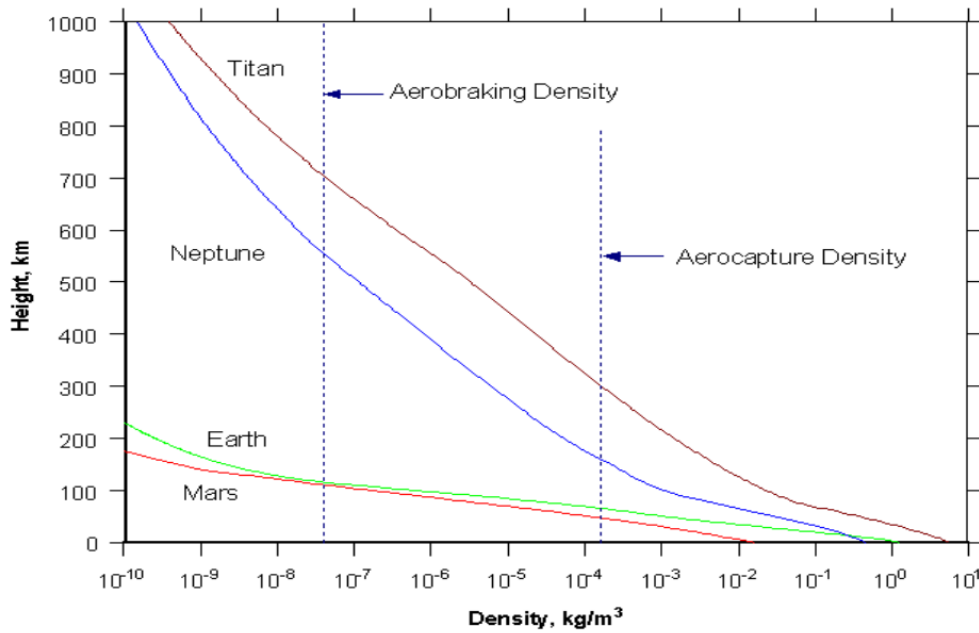


Figure 2.7: Atmospheric density comparison (Justus et al., 2003)

This gives aerobraking or aerocapture missions at Titan a larger safety margin for perturbations not predicted by a simulation and therefore decrease the risk of mission failure.

## 3 Description of the TAGAS Program Code

### 3.1 User Input

The user input is the first part of the program and is used to modify the simulation to be as close as possible to the planned mission. It is divided into rocket parameters and output parameters. The rocket parameters define the properties of the spacecraft. The parameters in Table 3.1 significantly impact the trajectory and must be adjusted before use to ensure realistic simulation results.

| Constant                   | Codename  | Value      | Unit           |
|----------------------------|-----------|------------|----------------|
| Total mass                 | $m_0$     | 5712       | kg             |
| Propellant mass            | $m_p$     | 2978       | kg             |
| Thrust                     | $thrust$  | 0.235      | N              |
| Hyperbolic excess velocity | $v_{inf}$ | 0.5 - 4.25 | km/s           |
| Specific impulse           | $I_{sp}$  | 3100       | s              |
| Drag coefficient           | $c_d$     | 2.2        | —              |
| Cross-sectional area       | $A$       | 11.05      | m <sup>2</sup> |

Table 3.1: Rocket parameters

Total mass ( $m_0$ ) represents the spacecraft's total mass that enters Saturn's sphere of influence. The propellant mass ( $m_p$ ) is the amount of propellant before any thrust maneuvers. Both masses are equivalent to the Cassini mission (Jet Propulsion Laboratory, 2019). Thrust ( $thrust$ ) is the maximum thrust the spacecraft's propulsion system provides. The propulsion system values are from the NEXT-C Gridded Ion Thruster electrical propulsion system (EPS) with low thrust but high specific impulse (Reckart, n.d.). Section 4.2 elaborates on the choice of the propulsion system. The hyperbolic

excess velocity ( $v_{inf}$ ) is the spacecraft's velocity relative to Saturn upon entering its sphere of influence. The specific Impulse ( $Isp$ ) measures the amount of thrust produced per unit of fuel. The drag coefficient ( $c_d$ ) is a dimensionless coefficient indicating how aerodynamically optimized the spacecraft is. The cross-sectional area ( $A$ ) describes the area of the spacecraft exposed to the atmosphere. The output parameters allow the user to display only those plots relevant to the mission analysis and to adjust the simulation duration. Table 3.2 shows the available settings in the program.

| Option                     | Codename          | Value      | Unit |
|----------------------------|-------------------|------------|------|
| Simulation duration        | tint              | 1500       | d    |
| Titan orbit crossings      | max_crossings     | 8          | —    |
| Plot trajectory            | Plot_Trajectory   | True/False | —    |
| Plot atmosphere            | Plot_Atmosphere   | True/False | —    |
| Plot data over time        | Plot_Values       | True/False | —    |
| Plot aerobraking altitudes | Plot_Comparison   | True/False | —    |
| Plot heat loads            | Plot_Heating_Rate | True/False | —    |

Table 3.2: Output parameters

Simulation duration (*tint*) is the maximum number of days the simulation runs. Titan orbit crossings (*max\_crossings*) provide an alternative way of limiting simulation time based on the number of times the spacecraft crosses Titan's orbit. Once the specified number of crossings is reached, the simulation stops. If the plot trajectory function is true, a plot of all simulated trajectory configurations is shown. Selecting a checkbox for a specific configuration will display its trajectory. A section of an exemplary plot with a single flyby is shown in Figure 3.1. The yellow circle represents Saturn, the light blue circle around Saturn represents Enceladus' orbit, the light red circle represents Titan's orbit, and the blue line represents the spacecraft's trajectory. The position of the spacecraft, Titan, and Enceladus at each time step can be shown by adjusting a slider below the plot. The entire figure, including Saturn's sphere of influence, is in the appendix at Figure A.2.

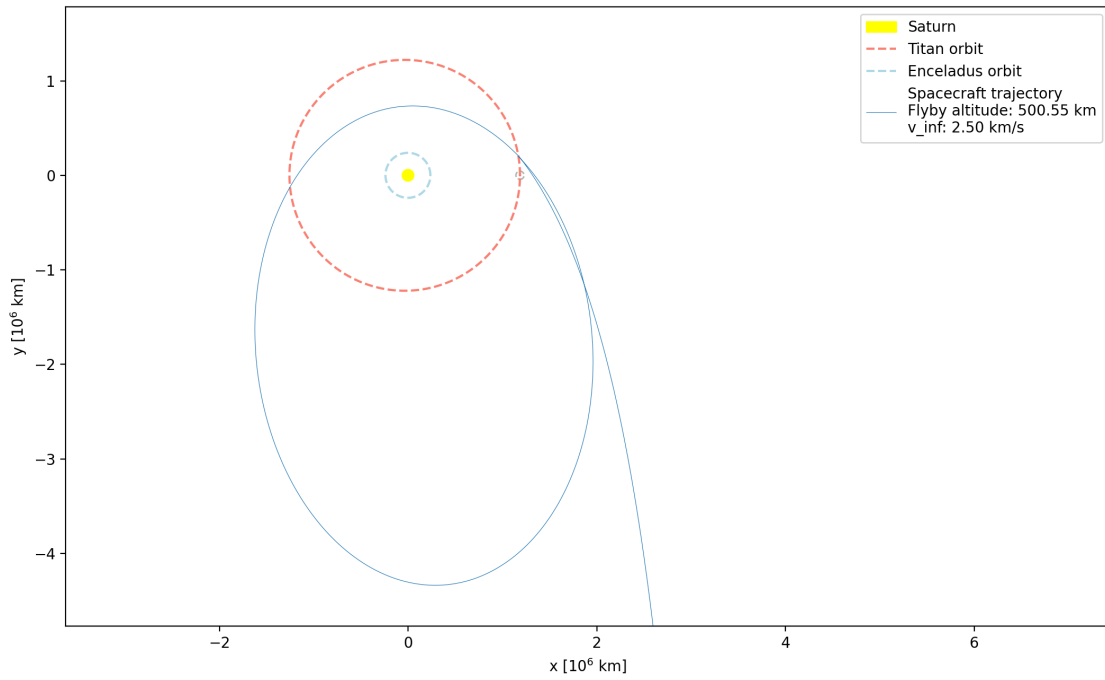


Figure 3.1: Trajectory plot

The atmospheric density based on altitude, shown in Figure 3.2, is displayed when the plot atmosphere option is set to true.

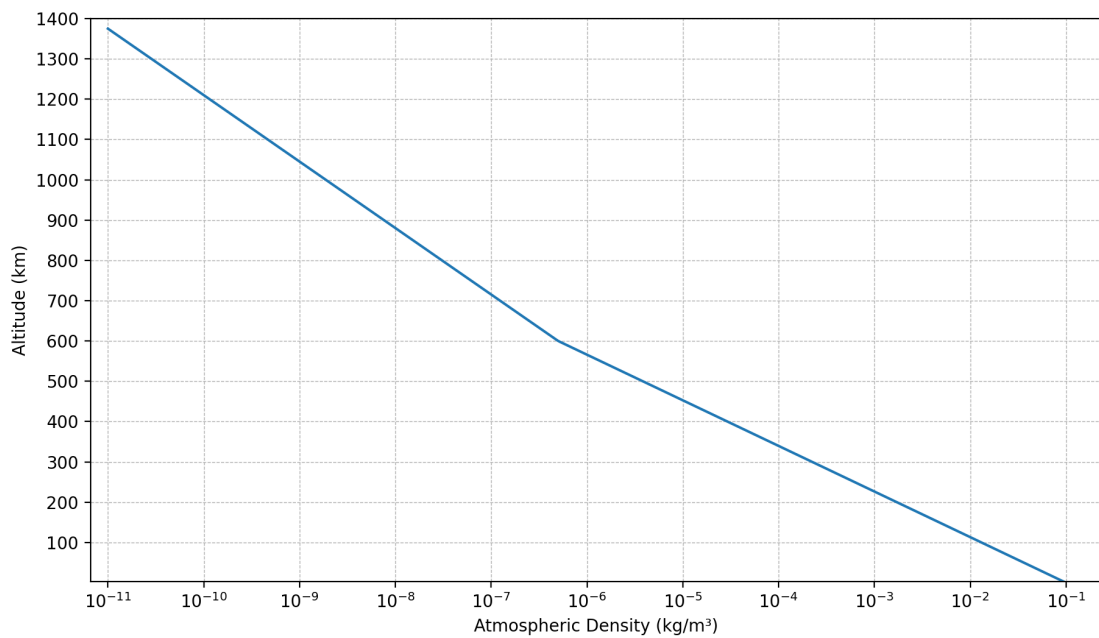


Figure 3.2: Atmospheric model plot

If the plot data over time function is enabled, the program displays various mission parameters over time. The parameters include orbital energy, distance to Saturn, distance to Titan, velocity, orbital period, and spacecraft mass. Periods during which the spacecraft is within Titan's atmosphere are shaded in light red. An example of the distance to Saturn data is shown in Figure 3.3.

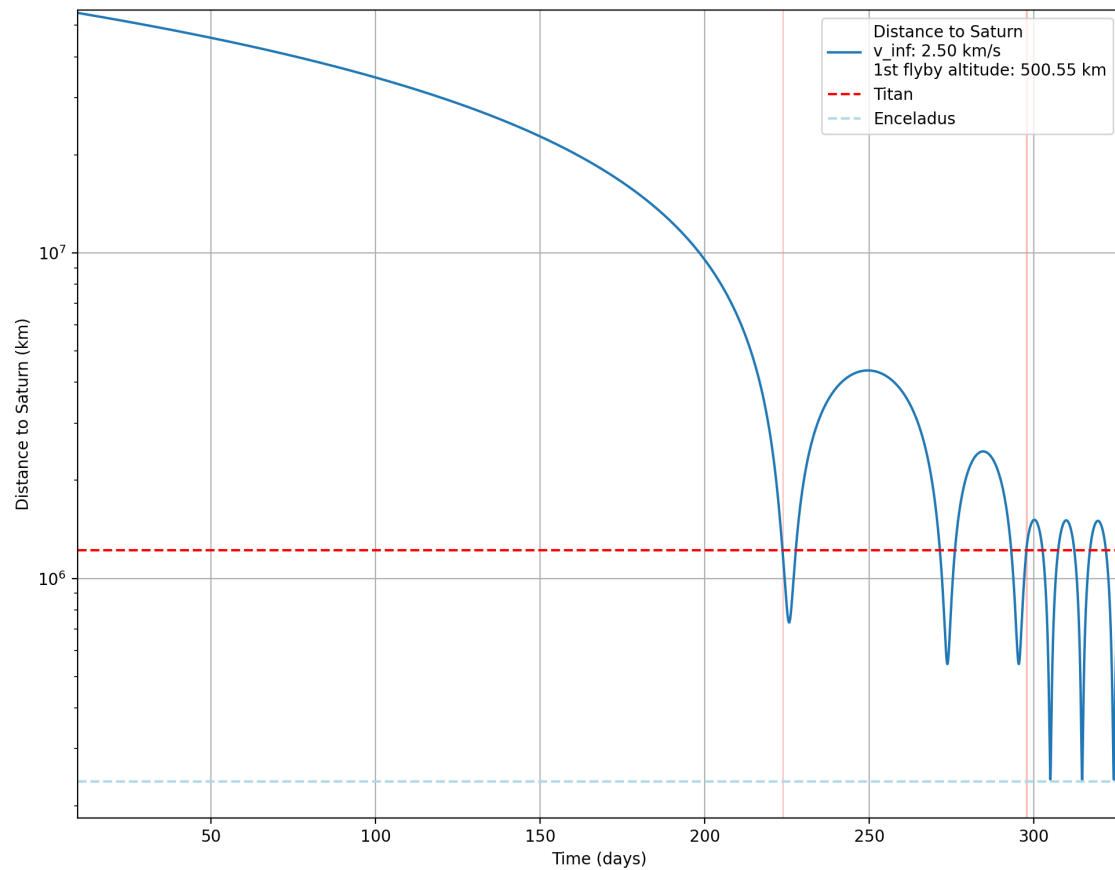


Figure 3.3: Data over time plot

The aerobraking altitudes plot, shown in Figure 3.4, illustrates the distance to Titan at the first orbit crossing, which leads to subsequent rendezvous on the following orbits for different arrival velocities. The data points, marked as circles, are sorted by the orbital energy after the flyby, and a polynomial is fitted through these points to hint at possible solutions between the simulated configurations.

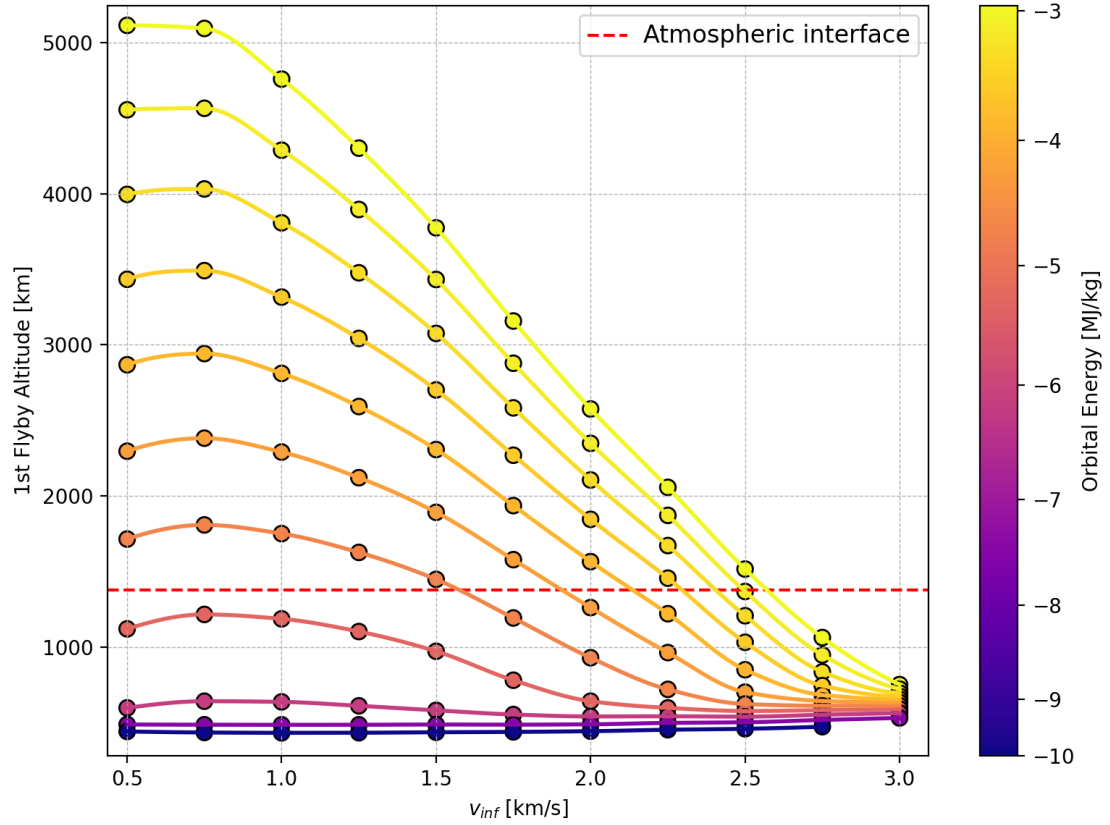


Figure 3.4: Possible altitudes for orbit capture at Titan

The heating rate plot displays the heat loads generated by atmospheric drag for each atmospheric phase generated by a configuration, as shown in Figure 3.5. For trajectories with more than one aerobraking maneuver, each atmospheric phase is different in color, so it can be easily separated which graph represents which heat loads. Furthermore, there is a distinction between radiative and convective heating because their thermal load can vary significantly depending on the aerobraking trajectory.

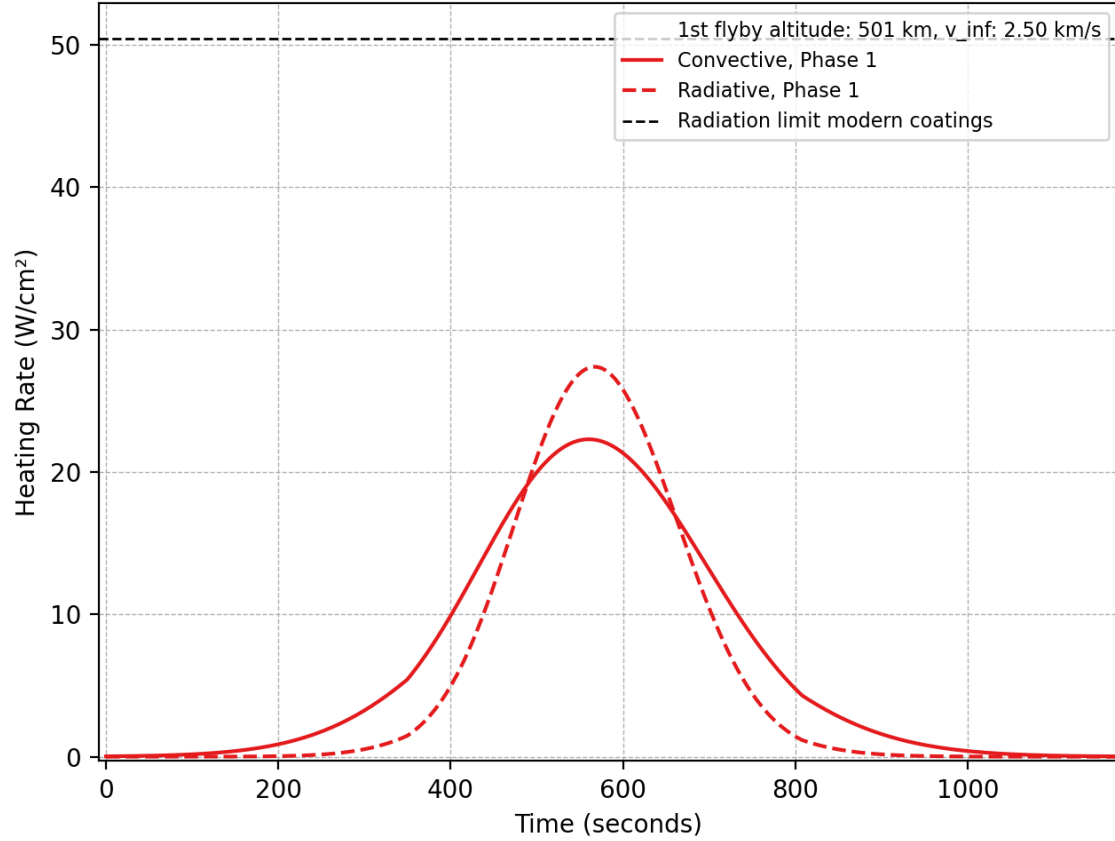


Figure 3.5: Heat loads during aerobraking

The two plotted graphs for each atmospheric phase are:

1. Convective heating rate: Using the Sutton-Graves equation with  $\rho$  as the atmospheric density,  $R_n$  as the radius of the aeroshell, so  $R_n = \sqrt{\frac{A}{\pi}}$  and the velocity  $v$  we get the convective heating rate in  $\text{W/cm}^2$  (Wright et al., 2012).

$$q_{\text{conv}} = 1.7415 \times 10^{-8} \sqrt{\frac{\rho}{R_n}} V^3 \quad (3.1)$$

2. Radiative heating rate: The radiative heating rate can be determined using the formula for uncoupled radiative heating and then adjusting for coupling using the following two equations.  $\kappa$  is an atmospheric specific constant that equals 2 for the atmosphere of Titan,  $k_{rad}$  is an empirical constant equal to 0.1 for this thesis (Wright et al., 2012).

$$\Gamma = 4 k_{rad} \quad (3.2)$$

$$q_{rad, unc} = \frac{k_{rad} \rho v^3}{10^4} \quad (3.3)$$

$$q_{rad} = \frac{q_{rad, unc}}{1 + \kappa \Gamma^{0.7}} \quad (3.4)$$

## 3.2 Program Preparation

This code section consists of the general planetary and orbital constants for the simulated Saturn-centered system with Titan and Enceladus, as found in Table 3.3. The radius, gravitational parameter, eccentricity, and semi-major axis are taken from the Horizons System by the Jet Propulsion Laboratory (Jet Propulsion Laboratory, 2022). The sphere of influence value is calculated using this data and the formula from Subsection 2.1.1 (Basics of Interplanetary Travel). Furthermore, there are accuracy parameters for the simulation, unit conversion, and variables to track the spacecraft's state during the simulation.



| Constant                             | Codename     | Value        | Unit                            |
|--------------------------------------|--------------|--------------|---------------------------------|
| Gravitational acceleration           | g0_earth     | 9.80665      | m/s <sup>2</sup>                |
| Gravitational parameter of Saturn    | mu_saturn    | 37931206.234 | km <sup>3</sup> /s <sup>2</sup> |
| Radius of Saturn                     | R_saturn     | 60268.0      | km                              |
| Saturn's sphere of influence radius  | R_SOI_saturn | 54807601.125 | km                              |
| Gravitational parameter of Titan     | mu_titan     | 8978.14      | km <sup>3</sup> /s <sup>2</sup> |
| Radius of Titan                      | R_titan      | 2575.5       | km                              |
| Titan's sphere of influence radius   | R_SOI_titan  | 43321.241    | km                              |
| Titan's semi-major axis              | a_titan      | 1221870      | km                              |
| Eccentricity of Titan's orbit        | e_titan      | 0.0288       | —                               |
| Orbital period of Titan              | T_titan      | 15.945421    | d                               |
| Gravitational parameter of Enceladus | mu_enceladus | 7.210367     | km <sup>3</sup> /s <sup>2</sup> |
| Radius of Enceladus                  | R_enceladus  | 252.3        | km                              |
| Enceladus' semi-major axis           | a_enceladus  | 238040       | km                              |
| Eccentricity of Enceladus' orbit     | e_enceladus  | 0.0047       | —                               |
| Orbital period of Enceladus          | T_enceladus  | 1.370218     | d                               |

Table 3.3: Planetary and orbital constants (Jet Propulsion Laboratory, 2022)

### 3.3 Atmospheric Model

The atmospheric model consists of two exponential functions that describe the density profile from Titan's surface up to an altitude of 1375 km, with a transition at 600 km. The base value on the surface of Titan is 0.1 kg/m<sup>3</sup>. This value does not match the value of 5.3 kg/m<sup>3</sup> found in the literature (Jet Propulsion Laboratory, 2019). The atmospheric density of Titan increases much faster below 100 km altitude. This altitude range is not relevant for an aerobraking maneuver for several reasons explained later in this thesis. The chosen surface density results in a very precise resemblance between the TAGAS atmospheric density model and the real mission data. At the connection altitude of 600

km, the density is  $5 \times 10^{-7} \text{ kg/m}^3$  and at the maximum altitude the value decreases to  $1 \times 10^{-11} \text{ kg/m}^3$ . These values result in the following approximation:

$$\rho(h) = \begin{cases} 0.1 \text{ kg/m}^3 e^{-\frac{h}{49.155860 \text{ km}}} & \text{for } 100 < h < 600 \text{ km,} \\ 5 \times 10^{-7} \text{ kg/m}^3 e^{-\frac{h-600 \text{ km}}{71.628085 \text{ km}}} & \text{for } h \geq 600 \text{ km.} \end{cases}$$

Figure 3.2 shows the atmospheric density over altitude. The atmospheric model is based on data from the Huygens Atmospheric Structure Instrument (HASI) (Jet Propulsion Laboratory, 2019). Subsection 4.1.1 (Validation of the Atmospheric Model) provides a more in-depth comparison.

## 3.4 Equations of Motion

The equations of motion section of the code contains four key functions. Three of them are helper functions:

1. `get_titan_position`: Given an initial point on Titan's orbit, this function determines Titan's position relative to Saturn at every simulation time step by solving Kepler's equation for the eccentric anomaly, then calculating the true anomaly. The starting location for Titan used in this thesis is at  $x = r_{p,\text{Titan}}$  and  $y = 0$ .
2. `get_enceladus_position`: Similar to the Titan function, this calculates Enceladus' position relative to Saturn at each simulation time step, again by solving Kepler's equation for the eccentric anomaly and then determining the true anomaly. The starting location for Enceladus used in this thesis is at  $x = -r_{p,\text{Enceladus}}$  and  $y = 0$ .
3. `get_initial_spacecraft_state`: This function provides the spacecraft's initial state for a given arrival velocity relative to Saturn in both Cartesian and polar coordinates. The spacecraft's initial state is a point on the edge of Saturn's sphere of influence. The lower the arrival velocity, the higher the x-axis offset needs to be for the spacecraft to arrive at the same location when crossing Titan's orbit. Moreover, it also passes along the change of the radius and angle relative to Saturn

at the starting point. Figure 3.6 illustrates how initial velocity variations affect the simulation's starting positions.

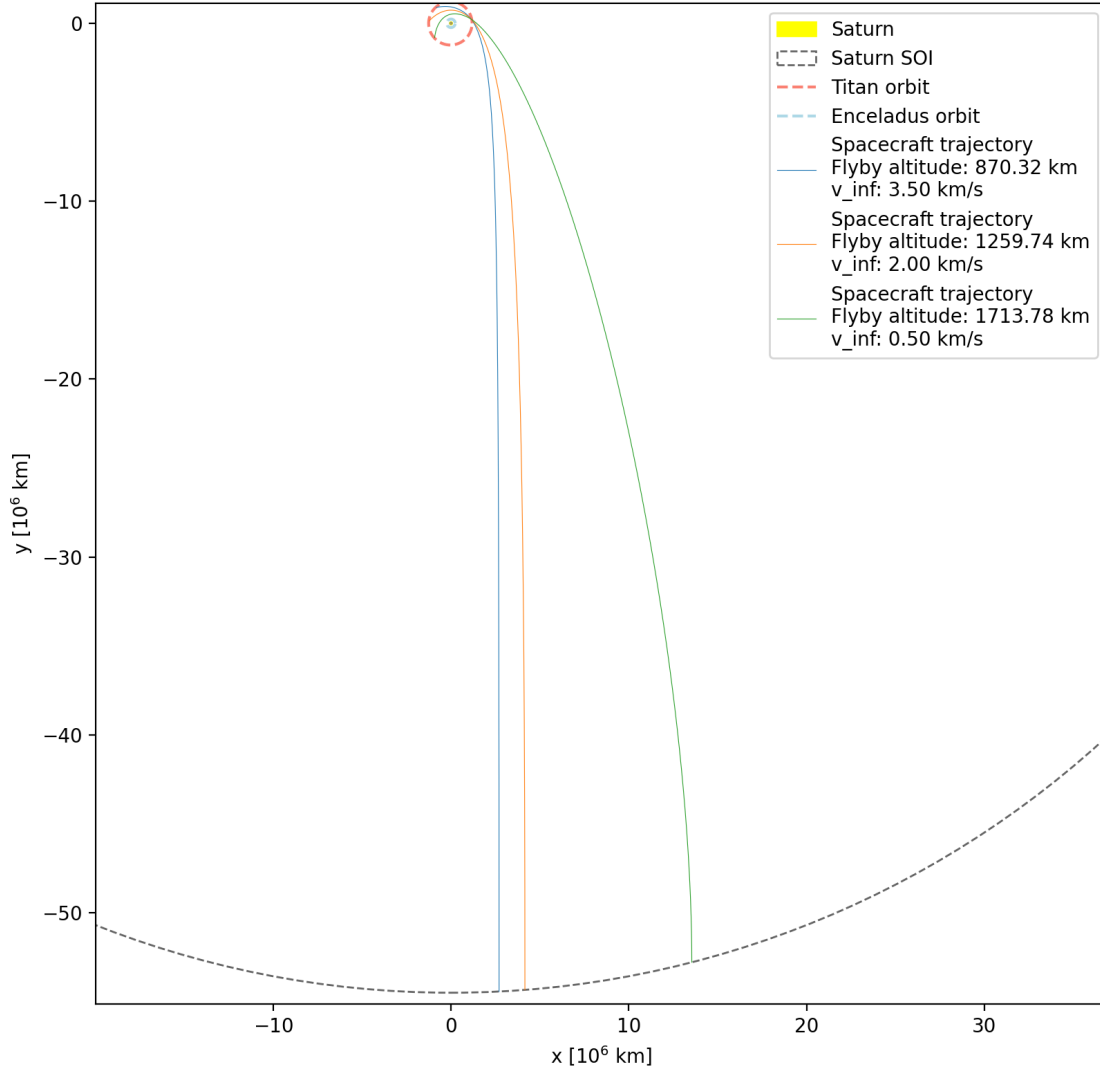


Figure 3.6: Initial spacecraft location for different arrival velocities

The fourth function is the main function, which calculates the spacecraft parameters listed in Table 3.4 at each step in the simulation. The throttle parameter describes the thrust behavior in a bang-bang control manner. In this simulation, the spacecraft's thrust system only allows for full or no thrust.

| Parameter                 | Codename     | Formula Symbol | Unit               |
|---------------------------|--------------|----------------|--------------------|
| Velocity                  | v            | $v$            | km/s               |
| Flight path angle         | alpha        | $\alpha$       | rad                |
| Thrust angle              | beta         | $\beta$        | rad                |
| Specific orbital energy   | E            | $E$            | MJ/kg              |
| Specific angular momentum | h            | $h$            | km <sup>2</sup> /s |
| Semi-major axis           | a            | $a$            | km                 |
| Eccentricity              | e            | $e$            | —                  |
| Apoapsis                  | apoapsis     | $r_a$          | km                 |
| Periapsis                 | periapsis    | $r_p$          | km                 |
| Throttle                  | throttle     | $\eta$         | 0/1                |
| Atmospheric density       | atmo_density | $\rho$         | kg/m <sup>3</sup>  |
| Airdrag                   | D            | $D$            | kN                 |

Table 3.4: Spacecraft status parameters

The main function uses the parameters of Table 3.5 to solve the following five differential equations and continuously update the state of the spacecraft:

$$\frac{dr}{dt} = \rho_r \quad (3.5)$$

$$\frac{d\varphi}{dt} = \rho_\varphi \quad (3.6)$$

$$\frac{d\rho_r}{dt} = r \rho_\varphi^2 - \frac{\mu}{r^2} - \frac{D}{m} \cos \alpha + \frac{\eta F_{Thrust}}{m} \cos \beta \quad (3.7)$$

$$\frac{d\rho_\varphi}{dt} = -2 \rho_r \rho_\varphi - \frac{D}{m} \sin \alpha + \frac{\eta F_{Thrust}}{m} \sin \beta \quad (3.8)$$

$$\frac{dm}{dt} = -\frac{\eta F_{Thrust}}{c_{eff}} \quad (3.9)$$

The equations are based on (Dachwald, 2023) and have been expanded by the term

$-\frac{D}{m} \cos \alpha$  for the aerodynamic drag that opposes the flight direction of the spacecraft.

| Parameter                              | Codename     | Formula symbol | Unit                            |
|----------------------------------------|--------------|----------------|---------------------------------|
| Spacecrafts radial position            | r            | r              | km                              |
| Spacecrafts angular position           | phi          | $\phi$         | rad                             |
| Change in spacecrafts radial position  | rhorr        | $\rho_r$       | km/s                            |
| Change in spacecrafts angular position | rhophi       | $\rho_\phi$    | rad/s                           |
| Gravitational parameter                | mu           | $\mu$          | km <sup>3</sup> /s <sup>2</sup> |
| Mass                                   | m            | $m$            | kg                              |
| Flight path angle                      | alpha        | $\alpha$       | rad                             |
| Thrust direction angle                 | beta         | $\beta$        | rad                             |
| Throttle                               | throttle     | $\eta$         | 0/1                             |
| Thrust                                 | thrust       | $F_{Thrust}$   | kN                              |
| Effective exhaust velocity             | ceff         | $c_{eff}$      | km/s                            |
| Airdrag                                | D            | $D$            | kN                              |
| Drag coefficient                       | c_d          | $c_d$          | —                               |
| Atmospheric density                    | atmo_density | $\rho$         | kg/m <sup>3</sup>               |
| Cross-sectional area                   | A            | $A$            | km <sup>2</sup>                 |
| Velocity                               | v            | $v$            | km/s                            |

Table 3.5: Equations of motion formula symbols

In this thesis applying the patched conics approximation means that for the majority of the mission, only Saturn's gravity is applied to the spacecraft. When the spacecraft enters Titan's sphere of influence, Saturn's gravity will be neglected, and Titan's gravity will be applied. Although this approach simplifies calculations, since only one gravitational force is considered at a time, it can lead to unstable results near the boundaries of Titan's sphere of influence. This instability is especially problematic when Titan and the spacecraft move in similar directions, causing the spacecraft to leave and re-enter Titan's sphere of influence multiple times because the direction and magnitude of the

force applied to the spacecraft changes rapidly. For the precise maneuver of this mission, constantly applying both gravity forces matches real-world dynamics more closely. Consequently, the gravitational effects of both Titan and Saturn are applied simultaneously to the spacecraft in this thesis.

## 3.5 Launch Delay Calculation

The Launch Delay Calculation aims to find a launch time that results in the spacecraft passing in front of Titan at a specific altitude out of a database. Further explanation of this first crossing distance and how it is generated is found in Section 3.6 (Aerobraking Altitude Adjustment). If the configuration of arrival velocity and first orbit crossing altitude has already resulted in valid results, the function will use the launch time from the database. If the database has no configuration entry, the launch delay calculation function starts an iteration process that follows a distinct logic. The function first simulates the spacecraft's trajectory until it crosses Titan's orbit for the first time. It then checks for the angular difference between the spacecraft and Titan at the crossing. A positive angular difference means that the spacecraft is ahead of Titan along its orbit. Hence, the launch delay needs to be increased to give Titan more time to travel toward the crossing point. Thus, a negative angular difference results in a decrease in launch delay. The adjustment in launch time is calculated based on the angular difference  $\Delta\varphi$  divided by the average angular velocity of Titan along its orbit, the mean motion  $n$ . Since Titan's orbit has a low eccentricity of 0.0288, its angular velocity does not vary significantly throughout its orbit, resulting in a fast angular adjustment until a threshold of up to  $2^\circ$  is reached. Within that, the function switches to an adjustment based on the distance error to the first flyby target altitude in km. The adjustment in launch delay now uses a coefficient of around 0.187 and multiplies it with the distance to the target altitude. The coefficient varies between 0.185 and 0.2 depending on the arrival velocity. However, the general logic of increasing the delay when the spacecraft's current altitude is higher than the target altitude and vice versa stays the same. The iteration process works with error-based adjustments, meaning larger errors lead to more extensive adjustments, resulting in fast convergence. The complete logic of the iteration process is illustrated in a flow diagram in Figure 3.7, and the section of the code is in Section C.1.

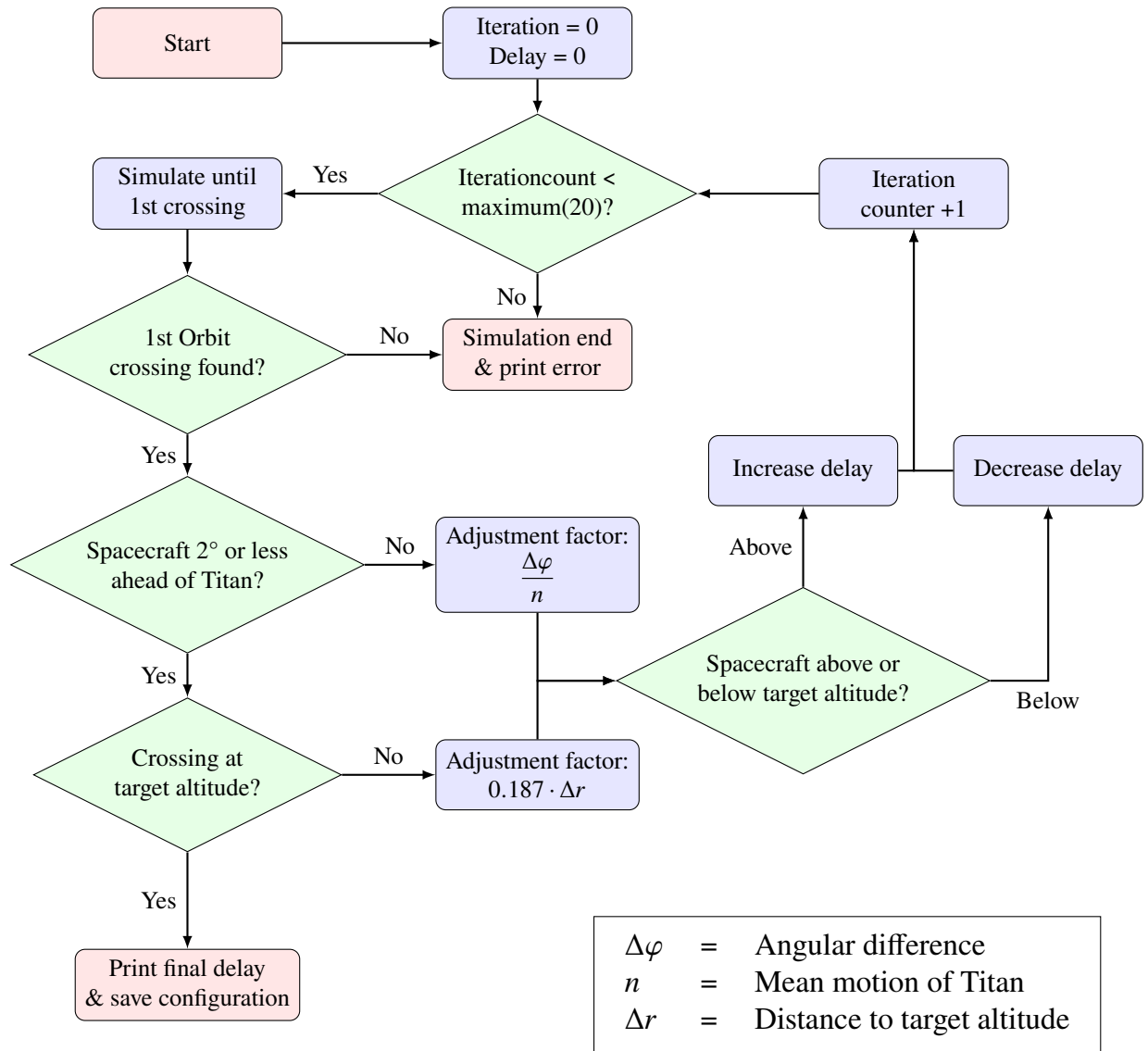


Figure 3.7: Logic of the launch delay iteration

Figure 3.8 provides an example of how the iteration process looks for values that are not yet listed in the database.

```

[0.50 km/s] Delay-Iteration 1 for r_p = 3740.911812 km: Spacecraft is 20.85 degrees away from Titan
Adjusting delay by 20.58 hours to 0.8575286511 days.
[0.50 km/s] Delay-Iteration 2 for r_p = 3740.91 km: Spacecraft is 1912.1872 km too far ahead of Titan.
Increasing delay by 5.53 minutes to 0.86136712 days.
[0.50 km/s] Delay-Iteration 3 for r_p = 3740.91 km: Spacecraft is 0.6636 km too far ahead of Titan.
Increasing delay by 0.12 seconds to 0.86136847 days.
ITERATION DONE FOR [0.50 km/s] | C1: 3740.91 km | C2: 1411898.83 km | C3: 2261.84 km | DELAY: 0.86136847 days

```

Figure 3.8: Delay iteration example

Once a suitable delay time is found for the chosen configuration, the program moves to Section 3.6 (Aerobraking Altitude Adjustment)

## 3.6 Aerobraking Altitude Adjustment

This function utilizes the launch delay found in Section 3.5 (Launch Delay Calculation) to simulate all configurations with their respective launch delays and first crossing distances over an extended duration. The user sets the exact simulation duration in Section 3.1 (User Input) and is limited either by the maximum number of Titan orbit crossings or the maximum integration time. During the simulation, the function tracks the spacecraft's relative position to Titan for every crossing of Titan's orbit. Its main goal is to adjust the first crossing distance to a value that leads to subsequent rendezvous of the spacecraft and Titan. More specifically, the database contains crossing values for as many rendezvous as the user requires. In practice, three to four Titan flybys have proven effective for lowering the spacecraft's orbit closely to Enceladus. Table 3.6 illustrates the structure of a typical database entry.

| Arrival Velocity | 1st Flyby | 2nd Flyby  | 3rd Flyby | Launch Delay |
|------------------|-----------|------------|-----------|--------------|
| 3.0 km/s         | 555.56 km | 1015.88 km | 584.52 km | 7.12066 days |

Table 3.6: Database entry

When creating a new entry in the database, the function first calls the Launch Delay function to get a valid launch time for the initial flyby distance. Since the launch delay calculation ensures the first crossing distance is at the desired altitude, the function skips that check and examines the spacecraft's position relative to Titan at the second flyby. The adjust aerobrake altitude function then, similarly to the adjust launch delay function, first checks for the angular difference between the spacecraft and Titan at the second crossing. There are two differences compared to the launch time function:

1. The adjustment factor varies depending on the crossing number. A change of 0.1 km in the first crossing altitude results in a difference of 900 km at the second crossing and  $57^\circ$  at the third crossing. Therefore, the adjustment factor, which



gets multiplied with the error, is not only based on the hyperbolic excess velocity, but also becomes lower with each crossing to ensure the next possible solution is found.

2. For even crossings, the adjustment is inverted. The apoapsis of the following orbit changes significantly with the altitude of every flyby. Hence, lowering the first orbit crossing distance will result in a bigger energy loss due to Titan's gravity and atmospheric drag. This leads to a smaller orbit and a shorter orbital period for the spacecraft. Thus, the spacecraft will be at the rendezvous point faster than before, increasing the distance to Titan at the second orbital crossing.

After adjusting the first crossing distance, the Launch Delay function is invoked again to ensure the desired changed altitude is met. This iterative process continues until the second flyby distance aligns with the user-specified value and the given tolerance of 1 km. The same procedure is then applied to the third flyby distance, continually readjusting the first crossing distance and launch delay until the third flyby also meets the required distance with a given tolerance of 30 km, as well as for the fourth flyby with a tolerance of 200 km. The tolerances are adjustable to the needs of the user. It is recommended to increase the tolerances with every consecutive crossing, as the required accuracy to meet the tolerances increases with every consecutive crossing, resulting in large computation times when looking for small tolerances at the fourth crossing or beyond due to very small simulation step sizes and error margins. The program prints a summary upon completion, as shown in Figure 3.9. This print statement contains the arrival velocity, all flyby distances, the final periapsis distance to Enceladus, and the launch delay used.

```
ITERATION DONE FOR [3.00 km/s] | C1: 555.5586265156 km | C3: 1015.8802680677 km | C6: 583.5230166957 km  
LAST PERIAPSIS DISTANCE TO ENCELADUS: 13760 km | DELAY: 7.12066324864928 days
```

Figure 3.9: Final print statement

Since the function continuously changes the first crossing distance and no thrust maneuvers occur until the end of the mission, the user-given values for the crossings before the last crossing will be changed. For example, once the second crossing distance is within the tolerance and the function starts adjusting for the third crossing, the second crossing altitude will change, as it solely depends on the first crossing altitude, which is being adjusted. However, the change in orbital energy by enabling a third crossing

significantly outweighs any potential losses of the increased second crossing altitude. Figure 3.10 shows that if the user initially specified the second flyby (C3) to be at 413 km, the program will search for a first crossing altitude (C1), resulting in the second flyby (C3) being at 413 km. However, to achieve a third flyby (C6) at 583 km, the first crossing (C1) needs to be adjusted again and, as a result, the second aerobraking altitude (C3) increases to 583 km. Despite the second flyby having a higher aerobraking altitude, Figure 3.9 shows that the final periapsis is almost 150,000 km closer to Enceladus now.

```
ITERATION DONE FOR [3.00 km/s] | C1: 555.5609947448 km | C3: 413.2046571134 km  
LAST PERIAPSIS DISTANCE TO ENCELADUS: 150830 km | DELAY: 7.12066324637848 days
```

Figure 3.10: Two vs. three flyby comparison

The complete logic of the iteration process for the aerobrake altitude adjustment is illustrated in a flow diagram in Figure 3.11, and the section of the code is in Section C.2.

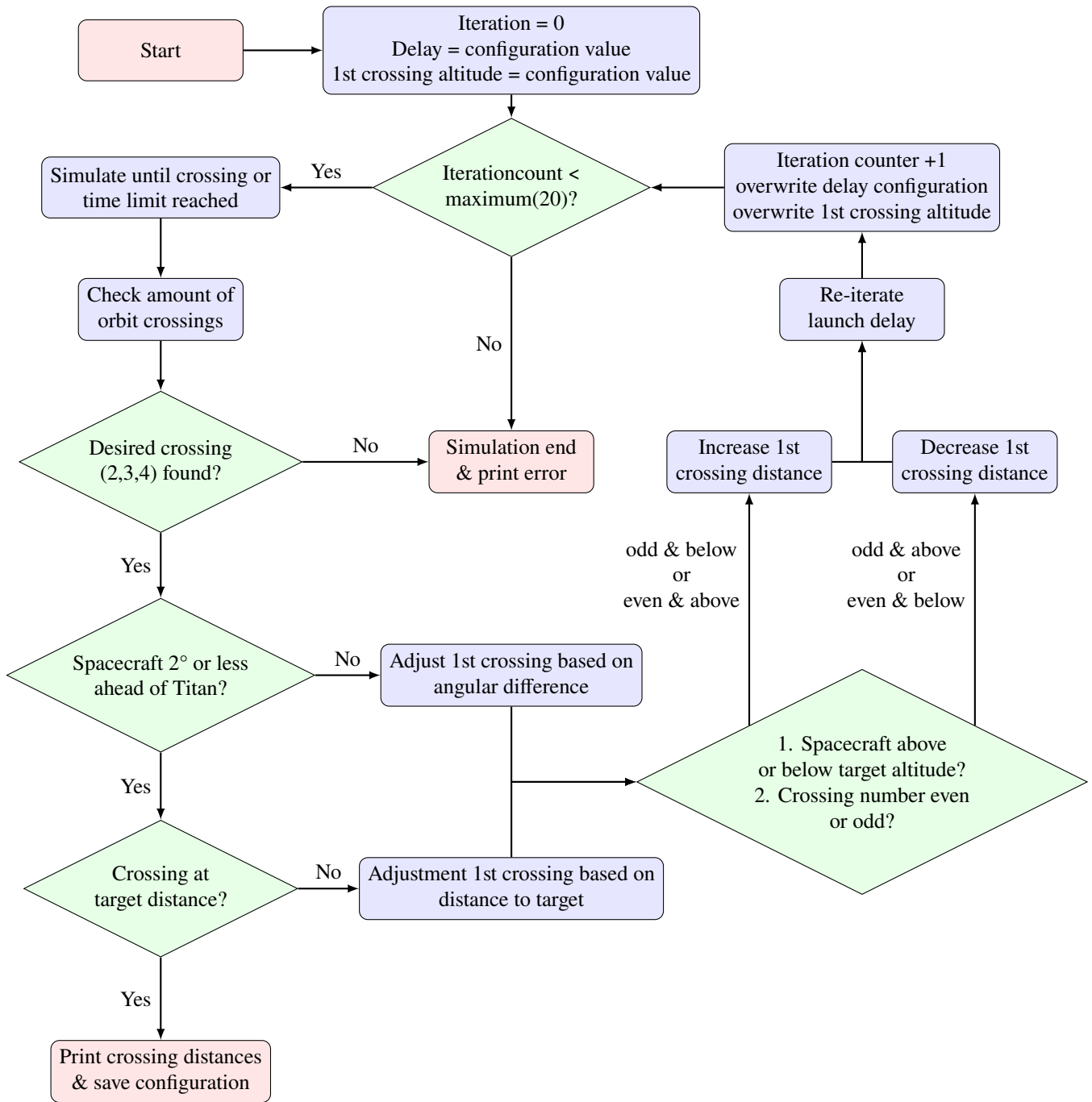


Figure 3.11: Logic of the aerobrake altitude iteration

# 4 Results

## 4.1 Validation of TAGAS

To ensure the reliability of the simulation results, the program developed in this thesis was validated by comparing its outputs with previously published studies and flight data in four critical areas:

1. The atmospheric model
2. Gravitational forces
3. Aerodynamic forces
4. Aerothermodynamics

Furthermore, this work builds on a previously developed Martian aerobraking program, which demonstrated less than a 1% error in predicting apoapsis changes when compared with data from the ESA ExoMars mission from 2017, using equivalent initial conditions (Kalscheuer, 2025).

### 4.1.1 Validation of the Atmospheric Model

As described in Section 3.3 (Atmospheric Model), the atmospheric model is based on data collected by the Huygens Atmospheric Structure Instrument (HASI) during the Cassini-Huygens mission from 1997 until 2017. HASI recorded Titan's atmospheric properties, such as altitude, temperature, pressure, and density, during its descent on January 14, 2005. Figure 4.1 illustrates how HASI continuously tracked these parameters during the descent.

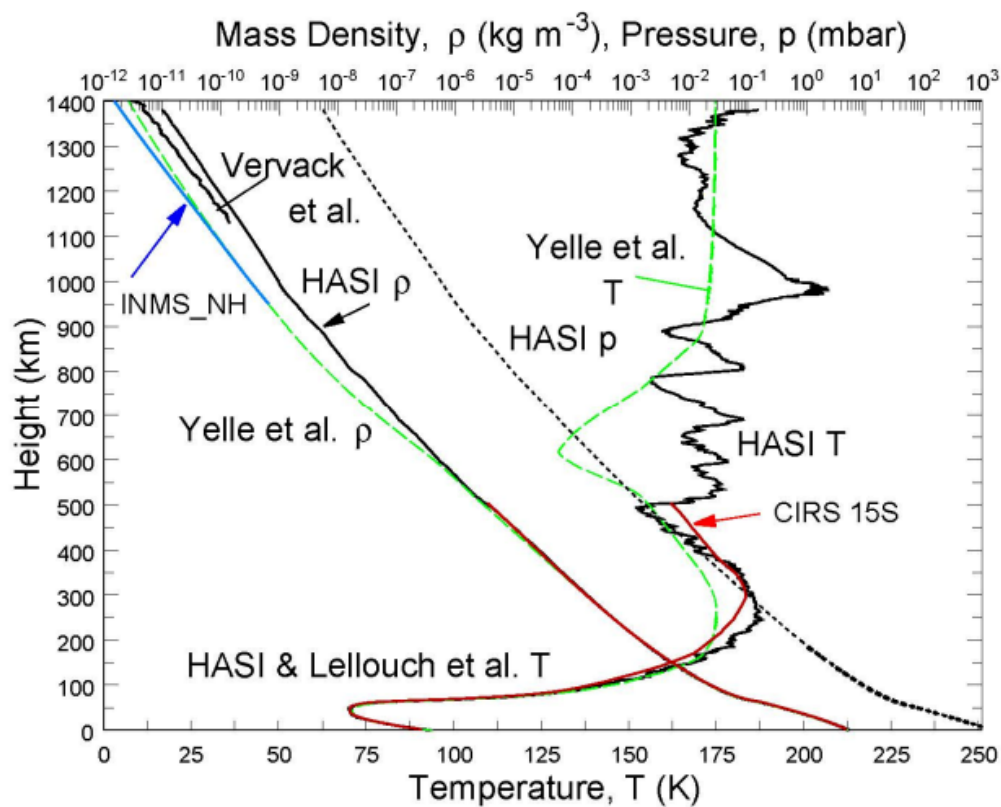


Figure 4.1: Atmospheric data from HASI (Jet Propulsion Laboratory, 2019)

Figure 4.2 compares the TAGAS atmospheric model (blue line) with the HASI data (red dots) in a single plot.

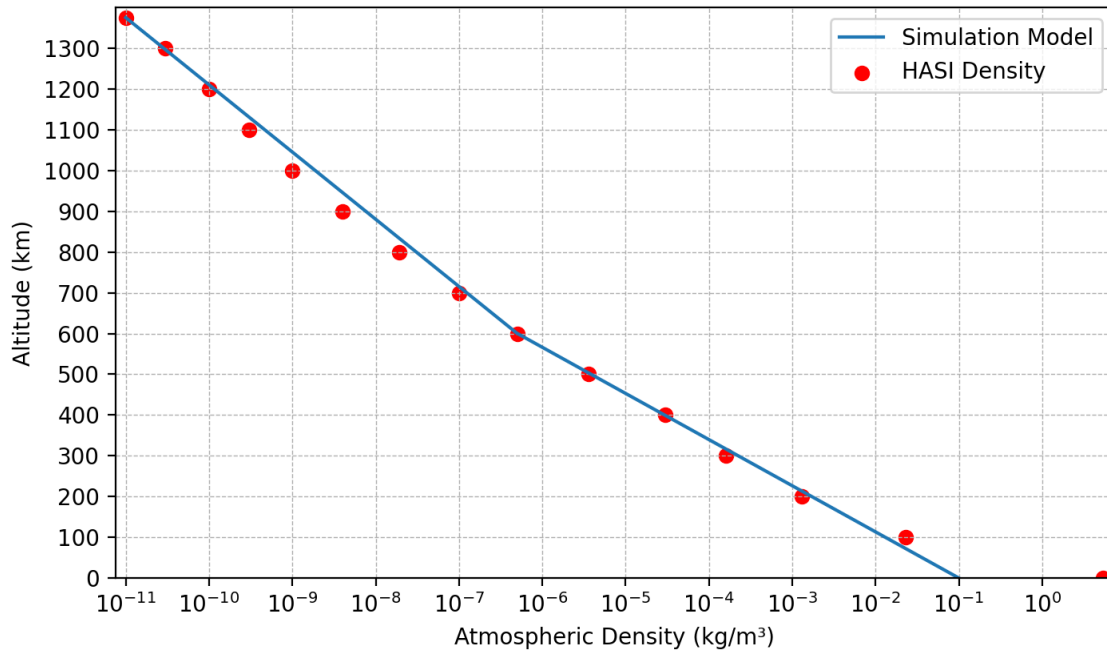


Figure 4.2: Comparison of TAGAS atmospheric model and HASI data

Notably, the model aligns precisely with the data at two key altitudes: at the outer boundary of the atmosphere (1375 km) and at 600 km. Although the model does not perfectly reproduce all measured values, it is slightly denser than the observations. This conservative bias ensures that the simulation does not underestimate atmospheric density, reducing the risk of mission failure. The model is structured in two sections to account for different atmospheric density gradients. Below 600 km, the increase in density is steeper than above this altitude. It should also be noted that below 200 km, the modeled density begins to fall below the observed values. However, any aerobraking maneuver below 200 km is impractical due to excessive heat loads at that altitude and the need for a safe distance to Titan's surface. Since HASI is the only source of atmospheric data for Titan, it is unclear whether the density varies with solar activity like the Martian atmosphere. Therefore, a conservative approach is advisable to ensure mission safety. Short thrust maneuvers allow necessary adjustments for discrepancies between the spacecraft's trajectory and the simulation. In summary, although the model is relatively simple, it is accurate enough to yield reliable results for mission analysis and planning, ultimately helping to ensure the safety of the mission while lowering fuel requirements.

### 4.1.2 Validation of Gravitational Forces

The simulation's treatment of gravitational forces was validated by comparison with an Aerocapture Design Study for a Titan Polar Orbiter (Nixon et al., 2016). To ensure comparable results, the program parameters are adjusted to match the ones from the study. Namely, the hyperbolic arrival velocity is set to 4.34 km/s, as Figure 4.3 from the study shows. Moreover, the B-plane offset is set so that the trajectory intersects with Titan's Orbit at the same point as shown in Figure 4.4 from the paper.

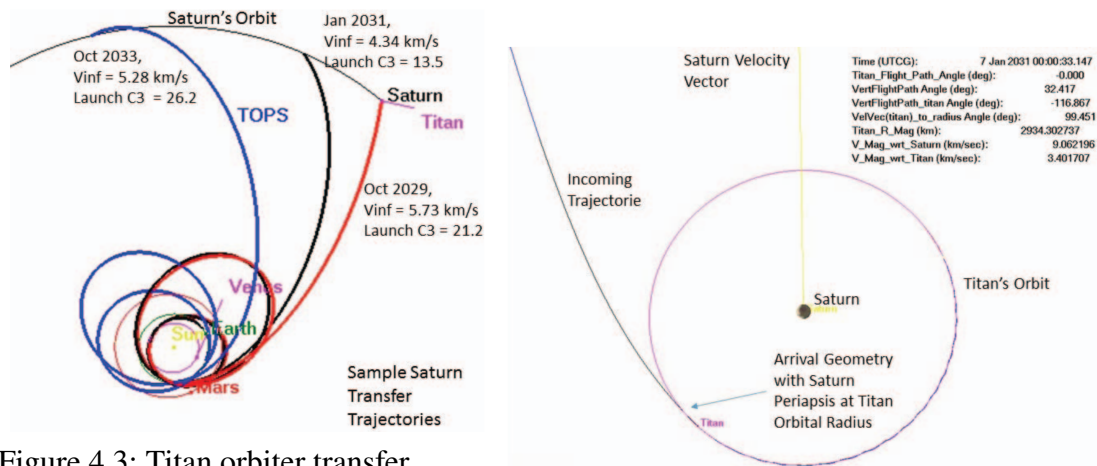


Figure 4.3: Titan orbiter transfer

trajectory (Nixon et al., 2016)

Figure 4.4: Titan orbiter arrival trajectory (Nixon et al., 2016)

At the point of intersection, the reference study reports a spacecraft velocity relative to Saturn of 9.062196 km/s, while the simulation yields 9.062055 km/s, as shown in Figure 4.5

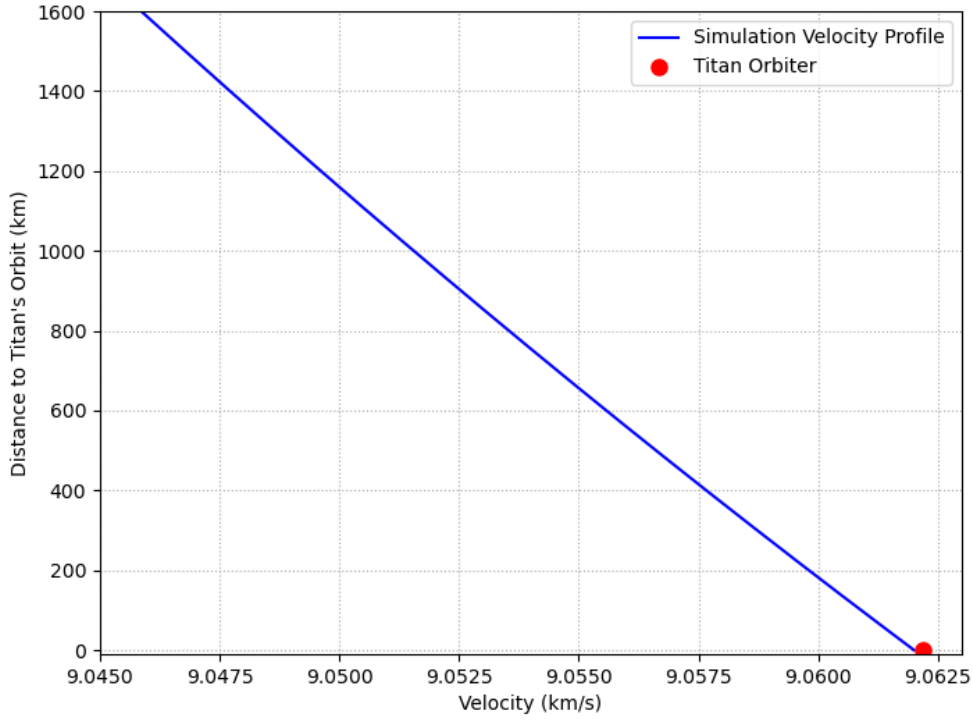


Figure 4.5: Spacecraft velocity when reaching Titan

This corresponds to an absolute error of 0.000141 km/s and a relative error of:

$$\text{Error}_{\text{rel}} = 1 - \frac{9.062055 \text{ km/s}}{9.062196 \text{ km/s}} = 0.001556 \% \quad (4.1)$$

Thus, this demonstrates excellent agreement between the simulation and the reference study.

### 4.1.3 Validation of Aerodynamic Forces

The aerodynamic forces were validated by comparing the simulation results with those presented in NASA's study "Aerocapture System Analysis for a Titan Mission" (Langley Research Center, 2006). Although the NASA paper primarily addresses aerocapture, its findings on atmospheric interaction across the entire altitude range are directly applicable



for cross-validation. The study identifies an optimal entry corridor at an angle of  $36.8^\circ \pm 1.0^\circ$  and uses an aeroshell with a diameter of 3.75 m, yielding a surface area of  $11.04466 \text{ m}^2$ . The aerocapture results from NASA are based on the Hybrid Predictor-corrector Aerocapture Scheme (HYPAS) algorithm. The algorithm has previously been extensively tested in simulations for aerocapture on Earth and Mars, and this study was the first to apply it to Titan. So, the HYPAS algorithm gives reliable results that have been tested and validated by NASA before for multiple celestial bodies (Langley Research Center, 2006). Figure 4.6 shows the spacecraft's velocity at different altitudes during the aerocapture maneuver.

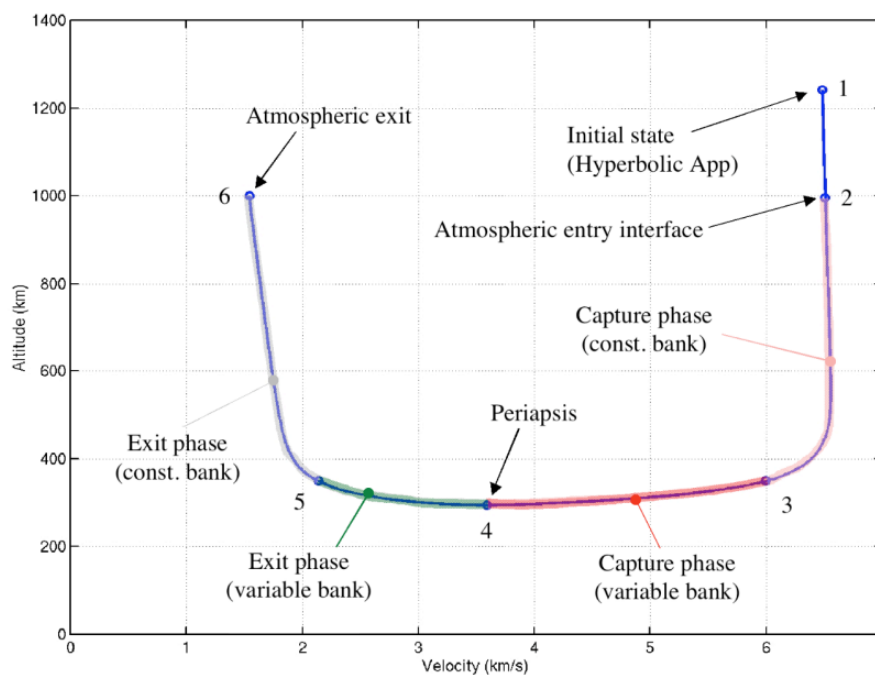


Figure 4.6: HYPAS aerocapture guide (Way et al., 2003)

The simulation was adjusted to match the initial conditions used in the NASA study to perform a direct comparison. Specifically, the atmospheric interface altitude was set at 1000 km, the angle of entry of the flight path was  $36.8^\circ$ , and the entry speed was 6.5 km/s relative to Titan. These values were chosen because they are the parameters used by the study to define the initial spacecraft state for the maneuver. With these parameters, the simulation produced a velocity profile during the aerocapture maneuver that can be directly compared to the one from the study. As shown in Figure 4.7, the simulation closely matches the HYPAS results, with deviations limited to approximately

20 km periapsis altitude and 0.1 km/s velocity. These minor differences are likely due to variations in the atmospheric model used, but overall, the strong agreement confirms that the simulation accurately captures the aerodynamic forces during the maneuver.

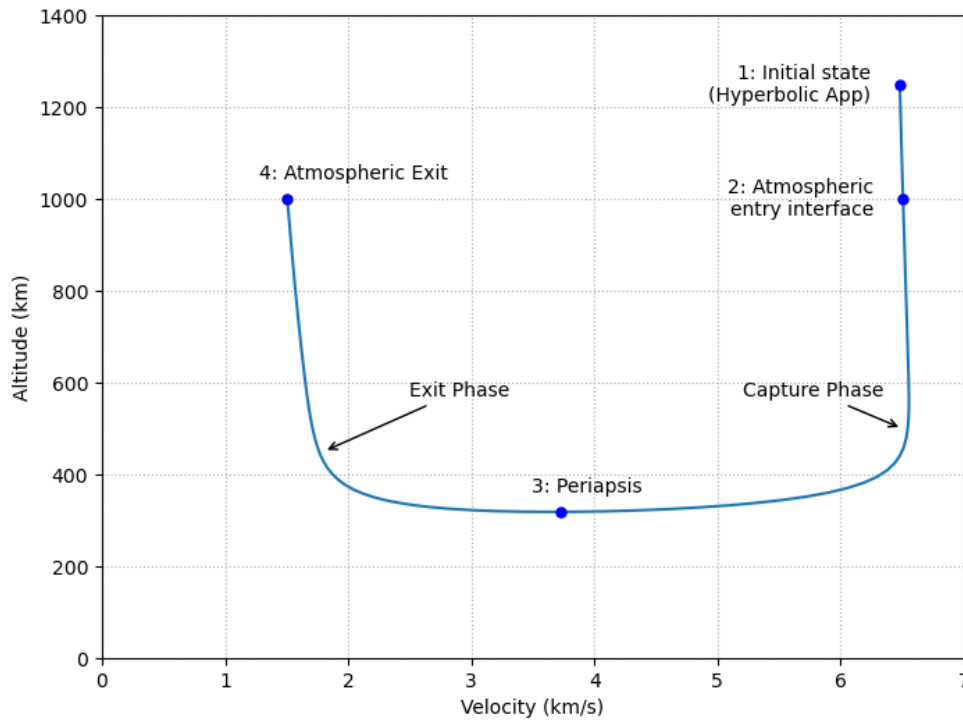


Figure 4.7: TAGAS aerocapture guide

This validation reinforces the model's reliability in predicting the spacecraft's behavior under aerodynamic forces, thereby supporting its applicability for mission planning and analysis.

#### 4.1.4 Validation of Aerothermodynamics

A comparison to a NASA study is made to ensure the heat loads projected by TAGAS are realistic. The green graph in Figure 4.8 represents the radiative and convective heat loads of the aerocapture maneuver for the nominal atmosphere at 6.5 km/s.

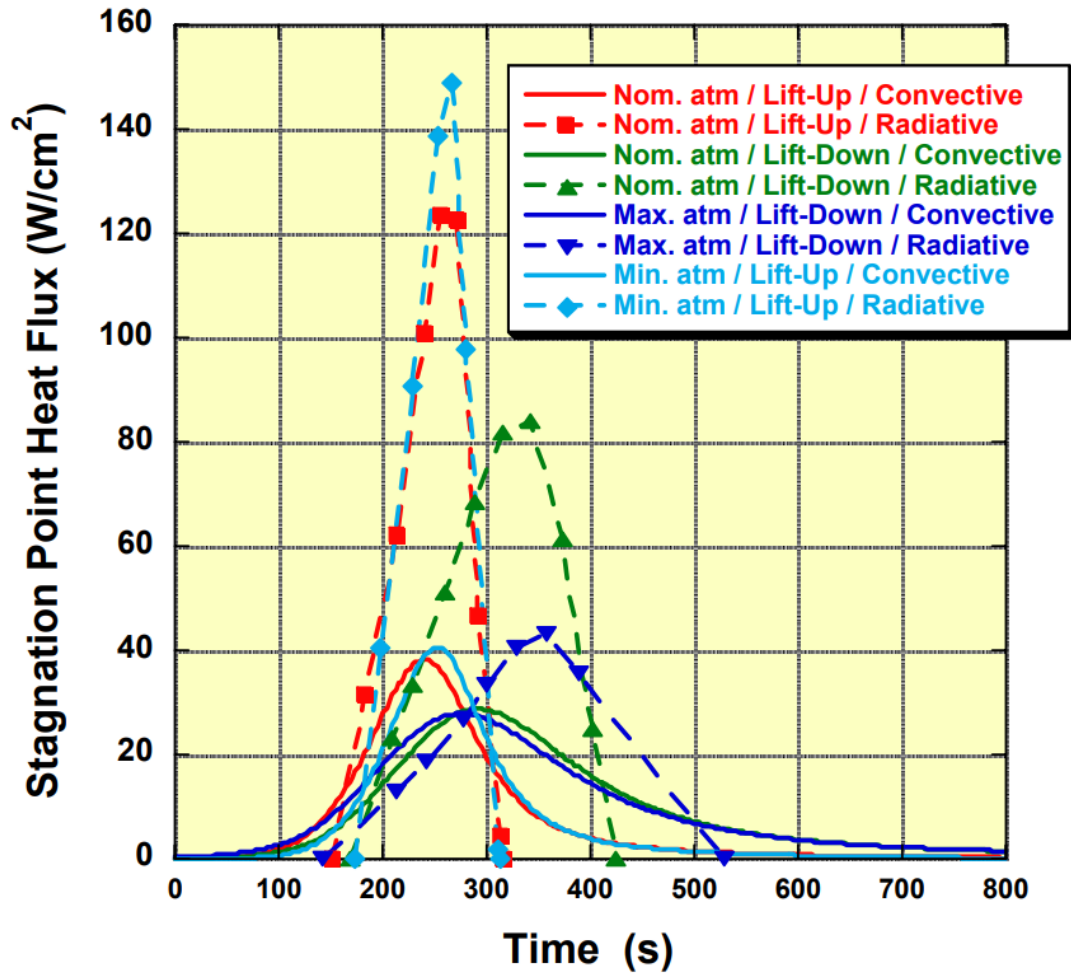


Figure 4.8: Heatflux during aerocapture according to NASA study (Laub, 2012)

In comparison, Figure 4.9 shows the heat loads simulated by the model used in this thesis. Although there is a noticeable difference, the reason for that is most likely a difference in atmospheric model and trajectory. A visual comparison of the atmospheric model and the altitude and velocity during the aerocapture is found in Section A.3. The simulated heat loads are within a range that matches the respective trajectory and atmospheric model.

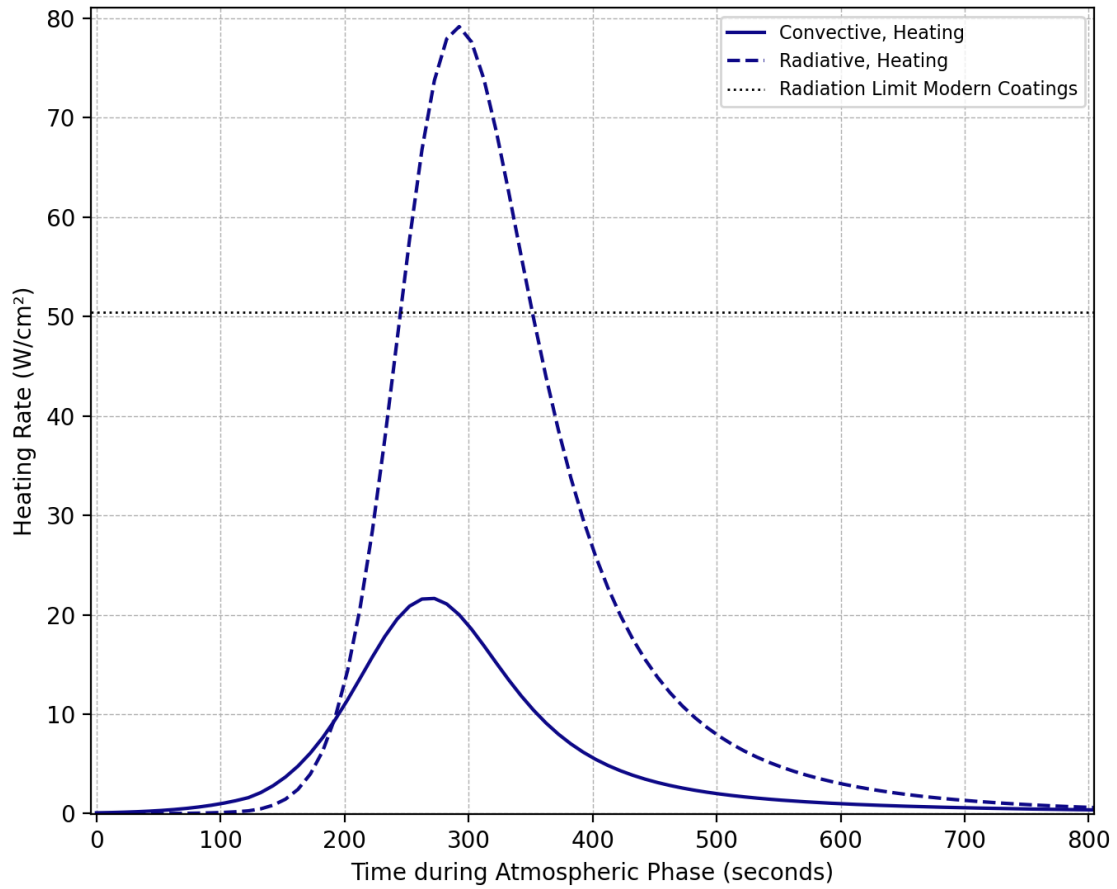


Figure 4.9: Heatflux during aerocapture according to TAGAS

## 4.2 Simulation Results

Analyzing possible flyby altitudes to stay within the Saturnian system results in finding multiple possible flyby altitudes for every hyperbolic excess velocity up to 4.25 km/s. The reduction in orbital energy for the first flyby at Titan needs to be strong enough to ensure that the spacecraft does not leave Saturn's sphere of influence. The apoapsis  $r_a$  has to be smaller than the radius of the sphere of influence from Saturn  $r_{\text{SOI\_Saturn}}$ . A high first flyby altitude results in a lower gravitational pull of Titan and, if it's within the atmosphere, a lower atmospheric density and a lower experienced air drag. As shown in Subsection 2.1.2 (Gravity Assist and Aerobraking), the gravitational pull decreases with the square of the distance between them, and the force of air drag is directly proportional

to the atmosphere's density. According to the atmosphere model shown in Figure 3.2, the density decreases exponentially with altitude. Hence, the experienced force slowing down the spacecraft heavily depends on the altitude to Titan. Therefore, it is expected that the higher the first flyby altitude, the larger the first apoapsis and, with that, the orbital period of the first orbit. This assumption is proven to be correct by Figure 4.10

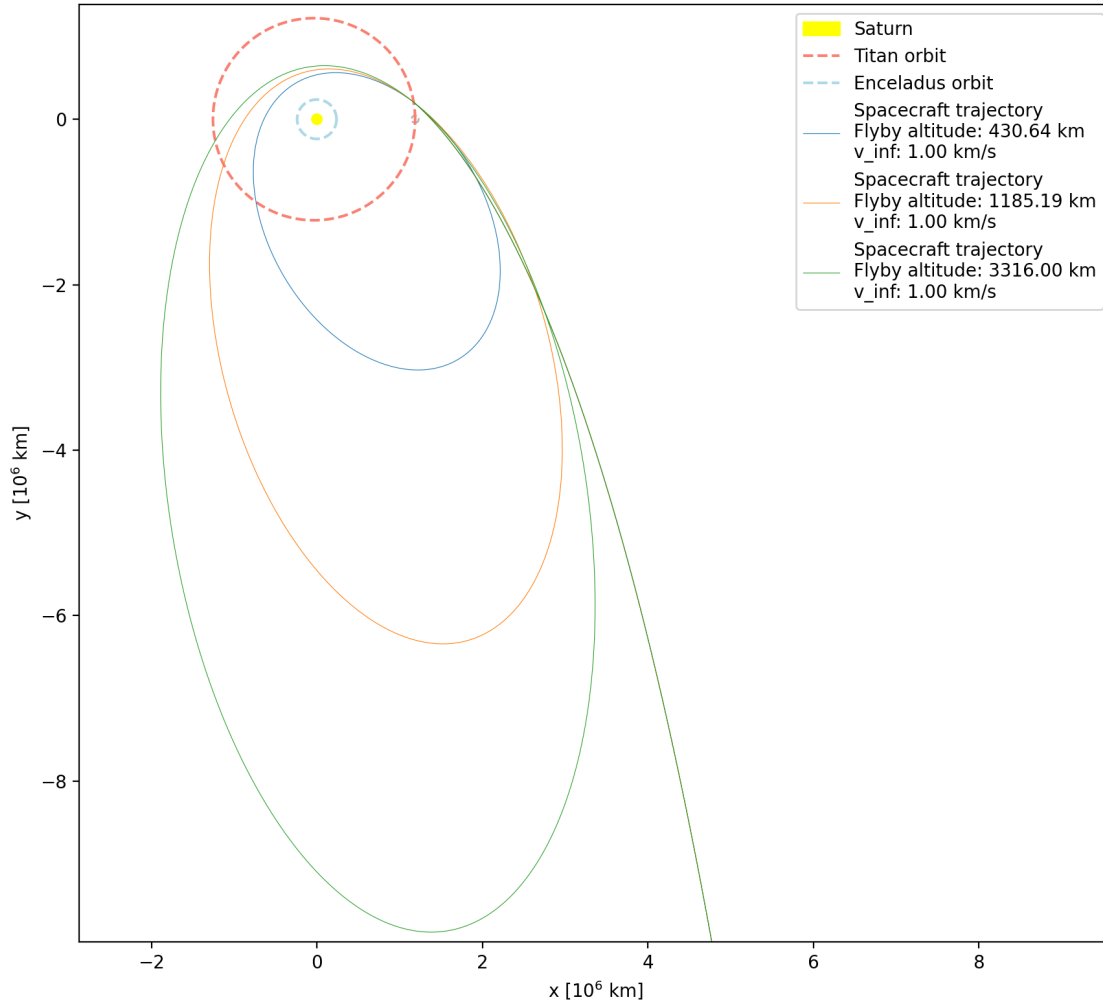


Figure 4.10: Comparison of apoapsis based on flyby altitude

Consequently, when optimizing for the time of flight, the mission analysis goal is to use the lowest flyby altitude possible. There is a limit on how low the first aerobraking altitude can be. Different factors can decide this limit. Firstly, it is essential to consider a safety margin that allows for minor deviations between the simulation and the real mission later. Checking for the lowest possible aerobraking altitude that does not result

in a crash into Titan due to the combined forces of gravity and aerodynamic drag delivers the result from Figure 4.11 for an arrival velocity of 0.75 km/s.

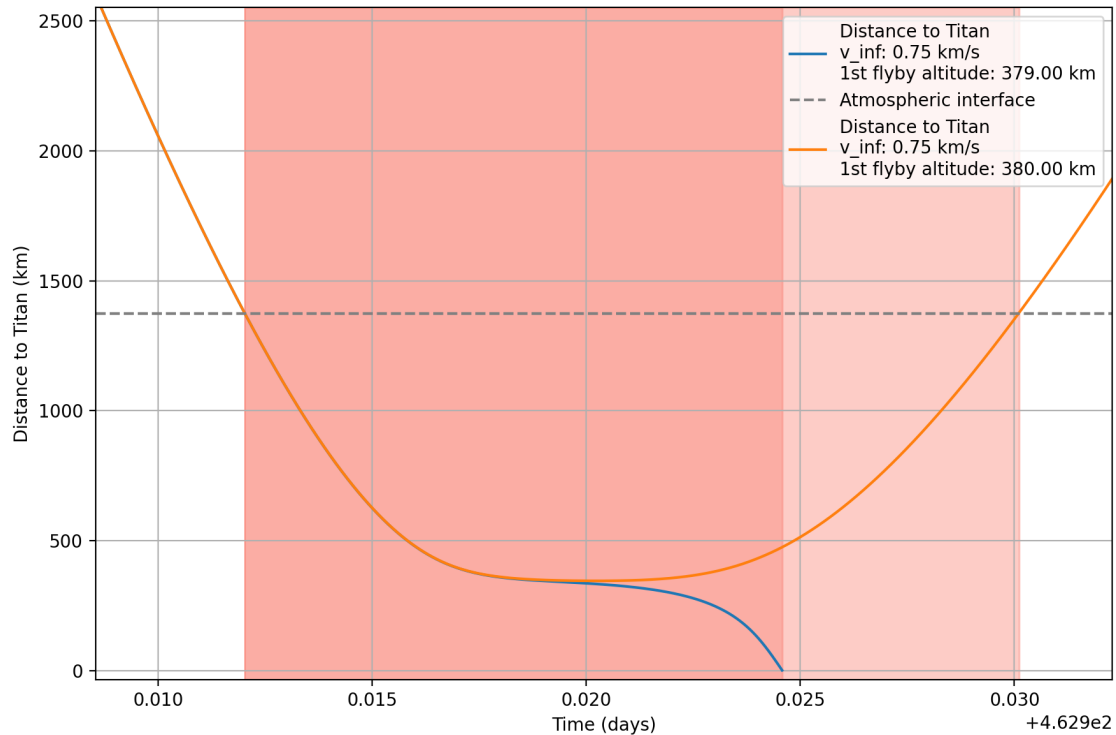


Figure 4.11: Lowest possible aerobraking altitude

Applying a safety margin of 10% to that altitude to ensure sufficient space for deviations from the simulation limits the aerobraking maneuver to a minimum first altitude of 418 km. Another constraint on the lowest possible altitude is the heat generated by friction between the spacecraft's cross-sectional area and the atmosphere. As discussed in Subsection 2.1.4 (Thermal Protection Systems), there are three primary ways to manage this thermal load. Thus, depending on mission constraints regarding the available mass to dissipate that thermal load, there is another limiting factor. However, the limit of the ablative heat dissipation method defines the spacecraft's overall thermal tolerance, as there is currently no superior heat shielding technology available. This limit is 1000 W/cm<sup>2</sup> (Laub, 2012). For the thermal load of that minimum altitude of 378 km, TAGAS predicts the results shown in Figure 4.12.

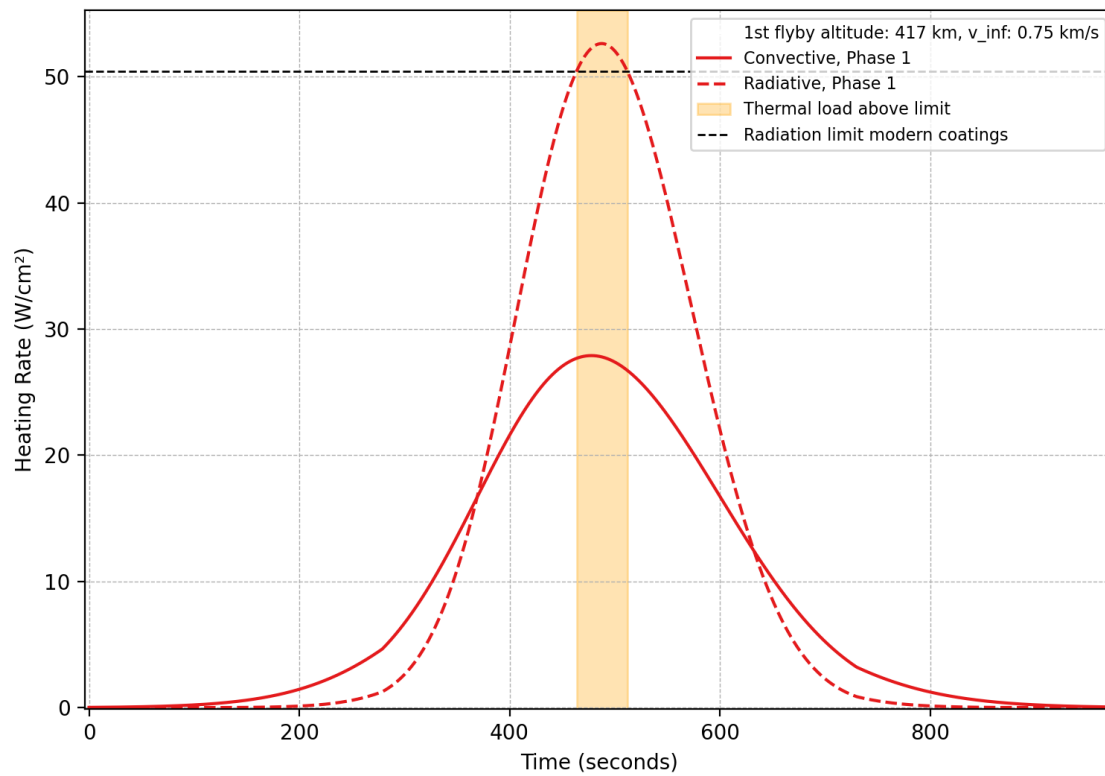


Figure 4.12: Thermal load at minimum altitude (incl. safety margin)

The orange highlighted area indicates that this thermal load barely exceeds the capacities of a radiative TPS. For a higher hyperbolic excess velocity of 4 km/s, the heating rate at the lowest possible altitude is  $95 \text{ W/cm}^2$ , making it evident that there is no aerobraking altitude where the thermal load will become the limiting factor in the maneuver for the arrival velocity in this thesis. Considering that a radiative thermal solution has significant advantages, it is reasonable to check for which altitudes this solution will be sufficient. Applying the safety margin to all arrival velocities and focusing on the lower altitudes, as these altitudes reach the target orbit faster, yields the following result from Figure 4.13.

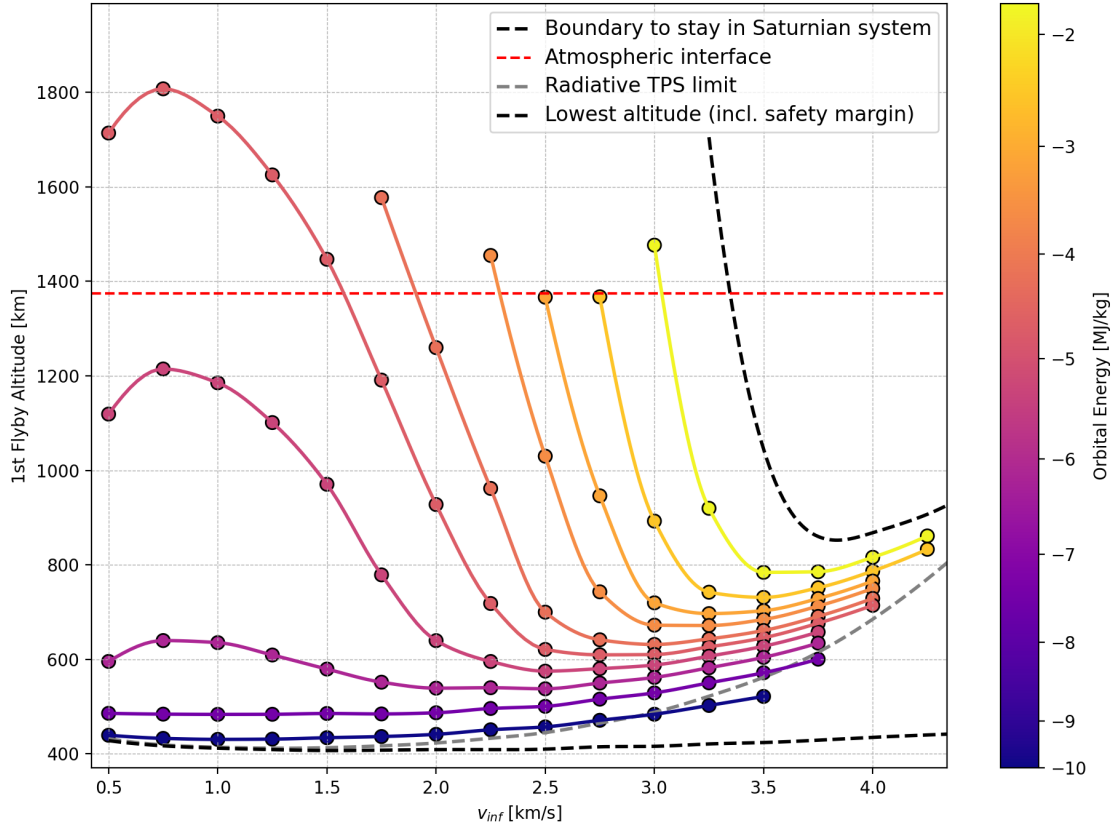
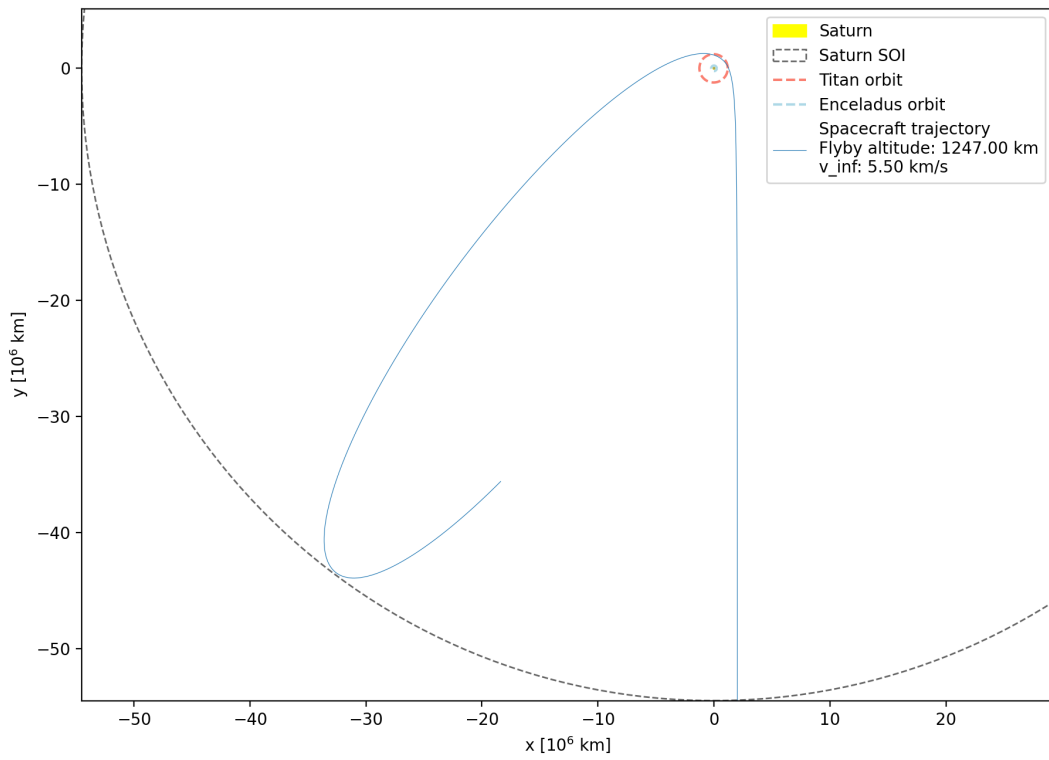
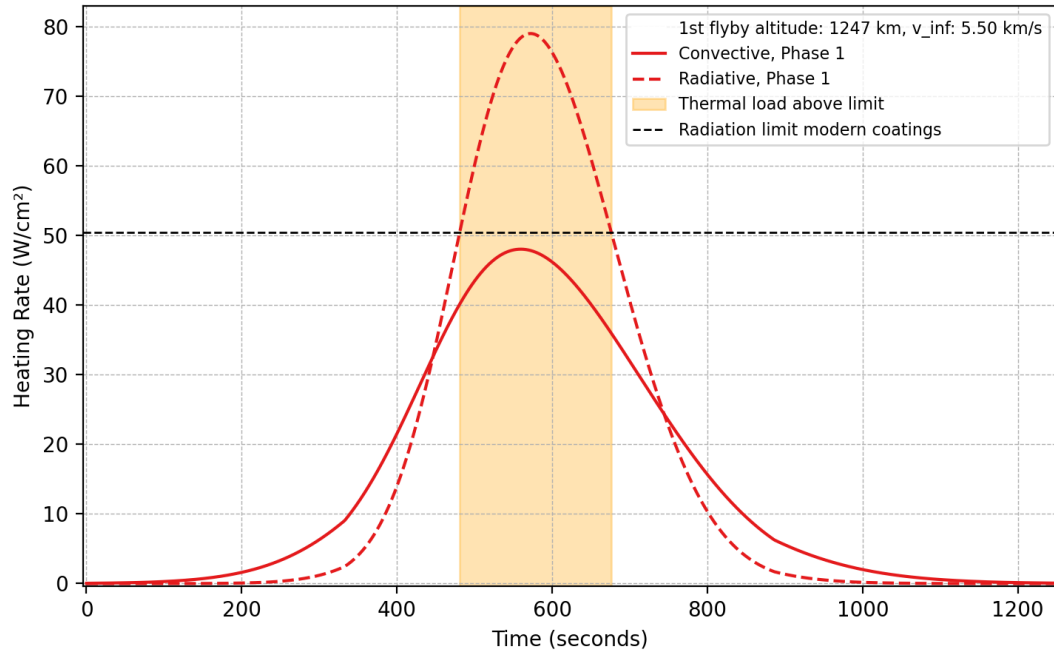


Figure 4.13: Possible flyby altitudes for orbit capture with altitude boundaries

Note that up to an arrival velocity of 2.0 km/s, this shows all possible aerobraking altitudes that result in a second rendezvous with Titan for further orbit adjustment without requiring a thrust maneuver. However, at higher arrival velocities, the possible aerobraking altitudes increase, as shown in Figure 3.4, and the figure becomes cluttered. Therefore, only specific data points are shown to illustrate the trend. Since the reduction in energy needs to be strong enough to keep the spacecraft in the Saturnian system after the initial flyby, an upper boundary on the first flyby altitude must exist. Pushing the approach velocity  $v_{inf}$  up to 5.50 km/s Titan's atmosphere still offers a velocity reduction large enough to keep the spacecraft in the Saturnian system. However, the heat loads undoubtedly require an ablative TPS, which requires a more thorough investigation and goes beyond the scope of this thesis. Using the highest aerobraking altitude possible to stay within the Saturnian system, the heating rate goes up to 80 W/cm<sup>2</sup>, as shown in Figure 4.14 and Figure 4.15.



Figure 4.14: Orbit for  $v_{inf} = 5.5$  km/sFigure 4.15: Thermal loads for  $v_{inf} = 5.5$  km/s

The highest arrival velocity  $v_{inf}$  that will still be within the Saturnian system after an aerobraking maneuver that does not exceed the  $50.4 \text{ W/cm}^2$  that modern coatings can withstand is  $4.5 \text{ km/s}$  with a minimum altitude of  $870 \text{ km}$  due to thermal loads and a maximum altitude of  $942 \text{ km}$  to lose enough energy to stay within the Saturnian sphere of influence, leaving a corridor of  $72 \text{ km}$  for potential aerobraking maneuvers, without the use of additional thrust maneuvers. One configuration that fits the elaborated criteria is shown in Figure 4.16. The spacecraft meets Titan three times along its trajectory at altitudes between  $500 \text{ km}$  and  $2011 \text{ km}$ , lowering the periapsis of the orbit to be within  $2000 \text{ km}$  of Enceladus' orbit.

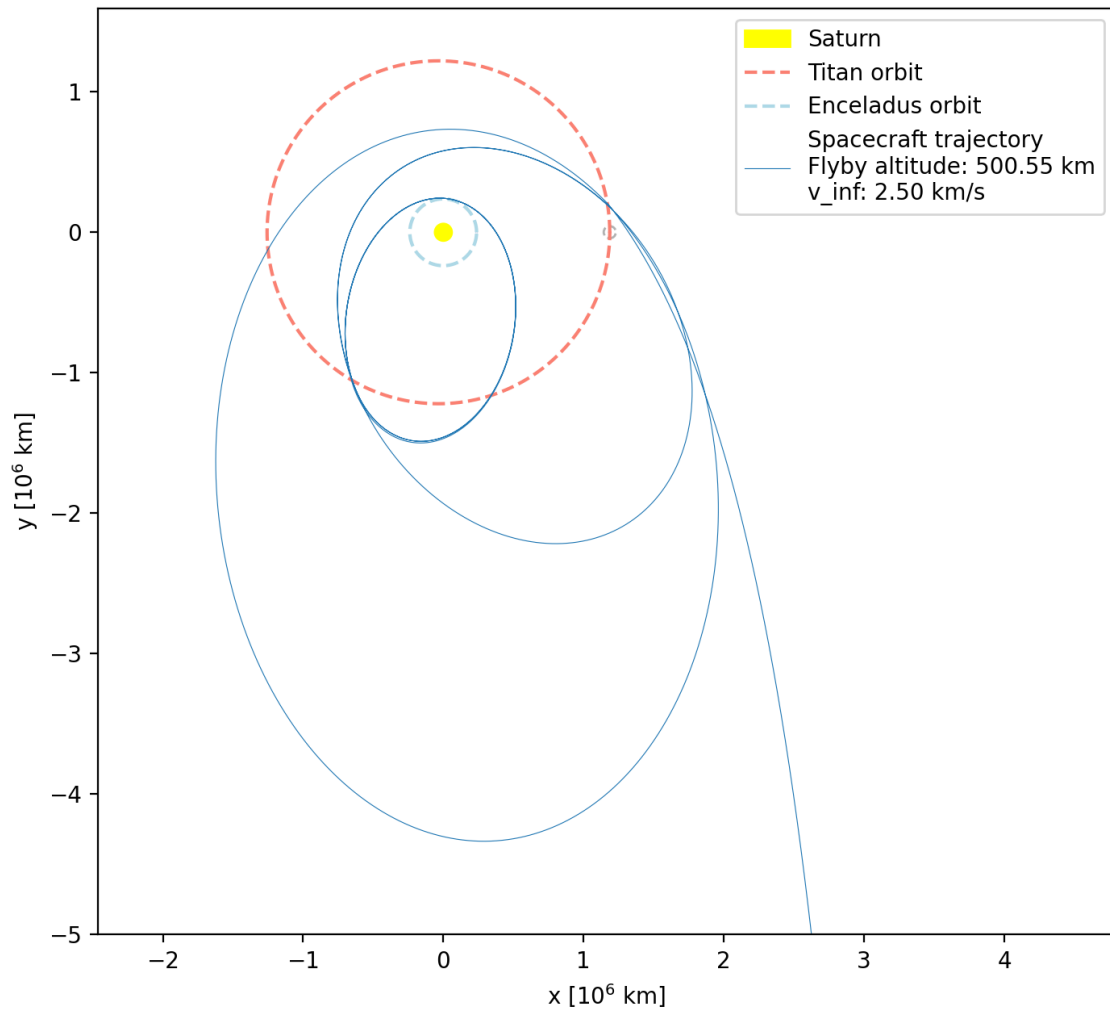


Figure 4.16: Final trajectory

According to Figure 4.17, both aerobraking phases are within the capacity of a radiative

thermal protection system using modern coating technology.

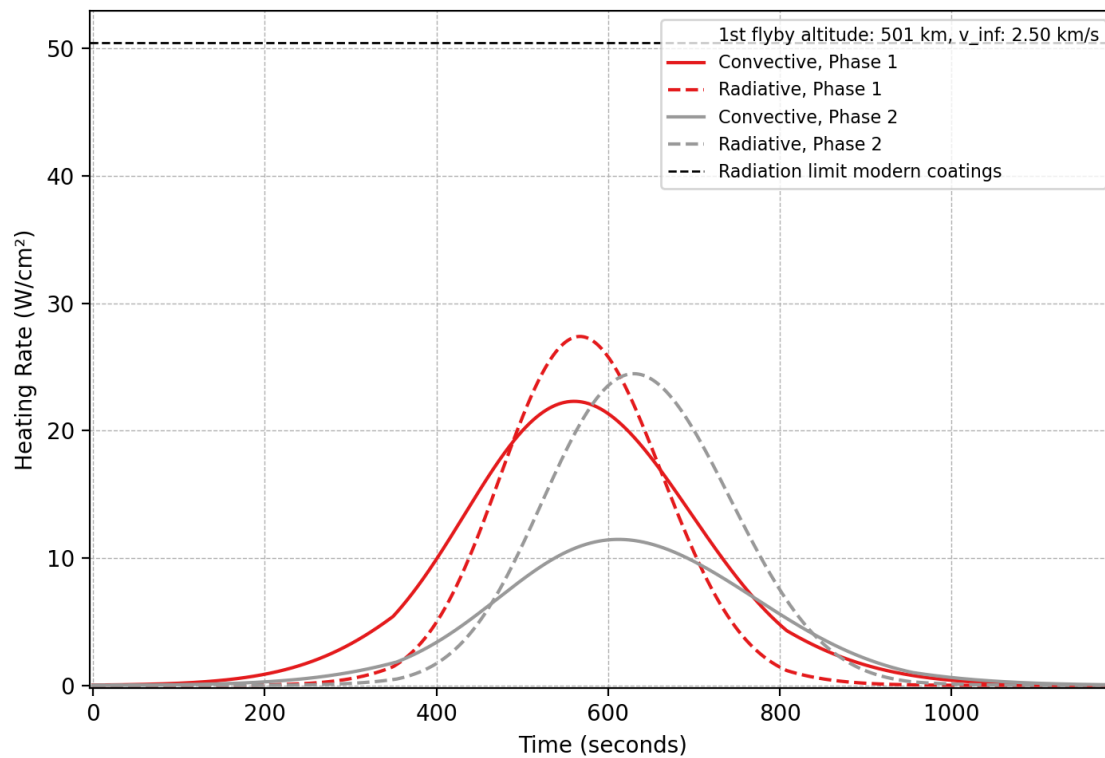


Figure 4.17: Thermal loads of the final trajectory

The last thing to achieve is a synchronous orbit with Enceladus. The orbital period of this configuration is shown in Figure 4.18. The orbital period in blue does not align with a multiple of Enceladus' orbital period, as shown in gray after the aerobraking phase, which is highlighted in light red. A thrust maneuver opposing the spacecraft's flight direction at the orbit periapsis lowers the orbital period to be synchronized with Enceladus. The spacecraft completes one orbit every seven Enceladus orbits and will meet Enceladus again after 9.592 days. The figure shows that the low thrust of an EPS is sufficient for this adjustment.

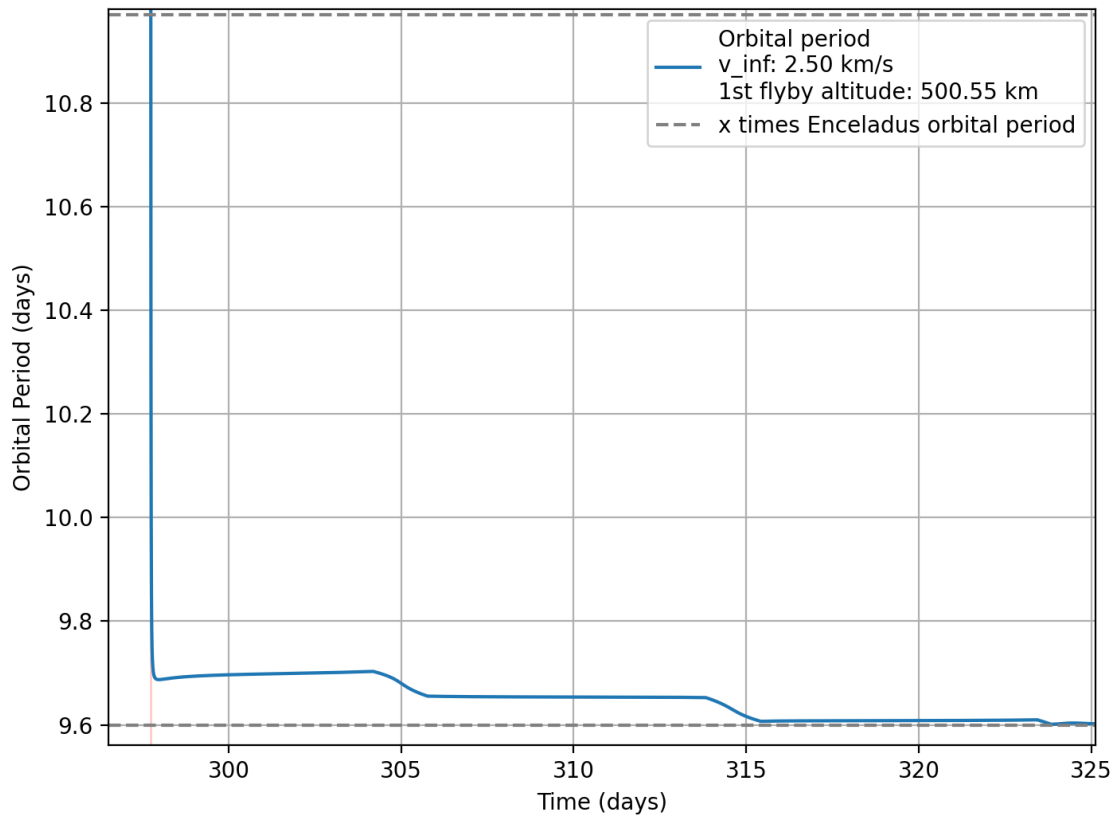


Figure 4.18: Orbital period of the final trajectory

It might be beneficial to lower the orbital period further to increase scientific up-time at Enceladus. A more significant thrust maneuver is required to achieve an almost circular orbit with a constant distance to Enceladus. Using the vis-viva equation from Subsection 2.1.1(Basics of Interplanetary Travel), the change in velocity required at the periapsis is 3.71 km/s. This maneuver would lower the velocity from 16.292 km/s to 12.584 km/s.

# 5 Conclusion

## 5.1 Summary

This thesis has demonstrated that utilizing Titan's gravity and atmosphere can be a key technique to realize fuel-efficient missions to the Saturnian system. The Titan Aerobraking and Gravity Assist Simulator developed for this thesis has been proven to provide reliable results for mission analysis. This data gives valuable insights into the possible utilization of aerobraking at Titan for future missions to the Saturnian system. In particular, for the high-priority target Enceladus. The TAGAS program has been validated against multiple NASA studies and real-world mission data to ensure simulation results that accurately predict interactions between the Saturnian system's gravitational forces, atmospheric drag, and thermal loads. The conservative nature of the atmospheric model ensures that the simulation successfully provides crucial safety margins for future mission planning. The thesis has shown how the current technological limits in the thermal protection field affect the minimum aerobraking altitude for different hyperbolic excess velocities and up to what velocity an aerobraking or gravity assist maneuver is sufficient and when thrusters will be required to capture the spacecraft into a stable orbit.

## 5.2 Outlook

While TAGAS provides an initial overview of the orbital dynamics and possibilities for aerobraking at Titan, there is ample opportunity to refine and extend the framework. Future enhancements could include:

1. Integration of real-time data: Incorporating NASA's Horizon system to supply real-time, real-world data for the positions of Titan and Enceladus around Saturn would allow for determining more accurate time frames for the spacecraft's arrival in the Saturnian system.

2. Enhanced atmospheric modeling: Using future research data to enhance the atmospheric model and taking a minimum and a maximum density for each altitude visualizes the impact of minor density deviations on the spacecraft's trajectory.
3. Three-dimensional approach: Extending the current two-dimensional model into three dimensions would enable the inclusion of orbital inclination and other spatial dynamics, thereby increasing the reliability of the results and offering a more comprehensive view of the trajectories.
4. Deviation sensitivity analysis: In the real-world application, the atmospheric density differs from the simulation. An analysis of the change in the spacecraft trajectory and timing for the second rendezvous with Titan for various deviations would provide insight into the required fuel to adjust the orbit to ensure further aerobraking opportunities.

Lastly, it is crucial to consider the simplifications used within the simulation. While the results are within small error margins, a more sophisticated simulation with the suggested enhancements should be made for further mission planning. Especially the sensitivity analysis allows for further research. Instead of spending fuel, it could be possible to adjust the cross-sectional during the aerobraking maneuver to adjust for differences in the expected and the measured atmospheric density. Considering that light takes between 67 and 90 minutes to travel from Earth to Titan, this system would most likely need to be autonomous because of the communication delay between the spacecraft and any ground station operating it. By addressing the mentioned improvement opportunities, future research can further optimize gravity assists and aerobraking, ultimately paving the way for the first spacecraft to land on Enceladus.

# A Additional Figures

## A.1 Orbit of All Saturnian Moons

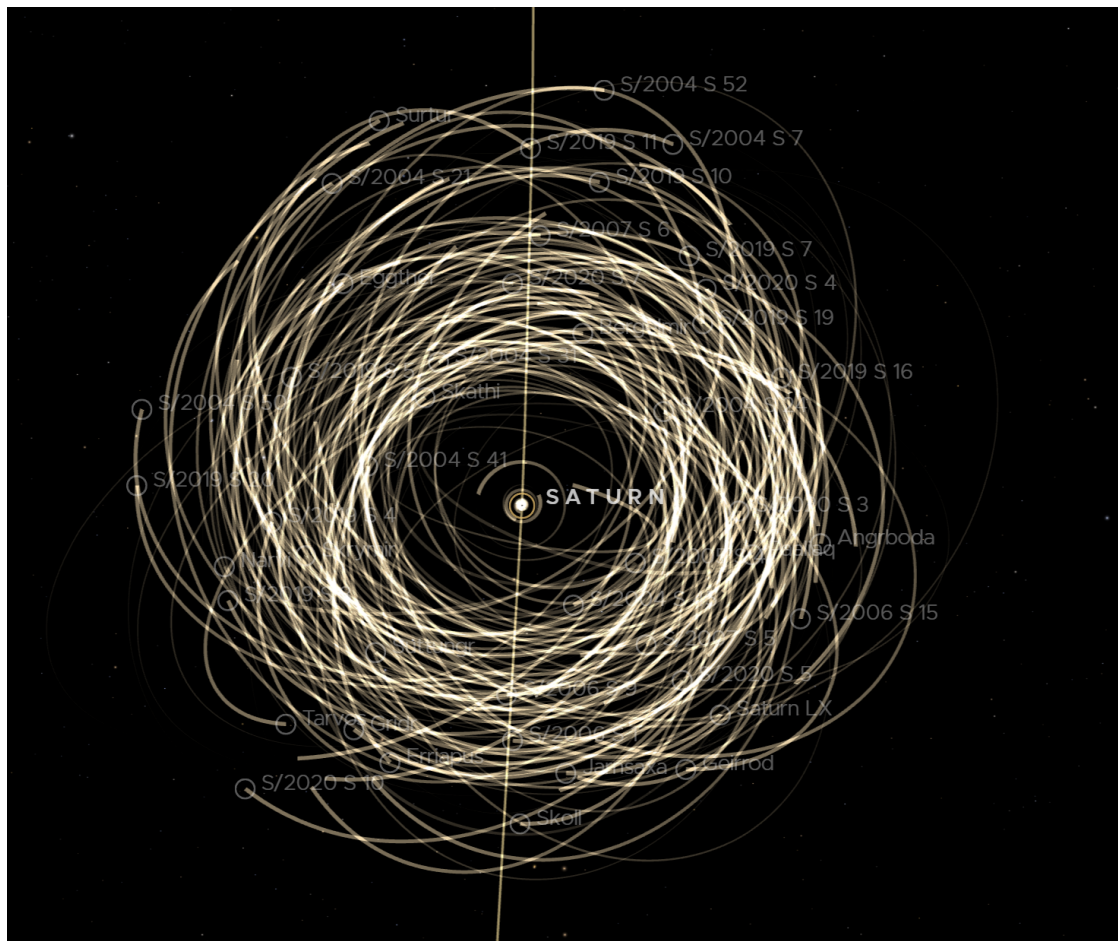


Figure A.1: Saturn and all known moons (Barnett, 2025)

## A.2 Full Trajectory Plot

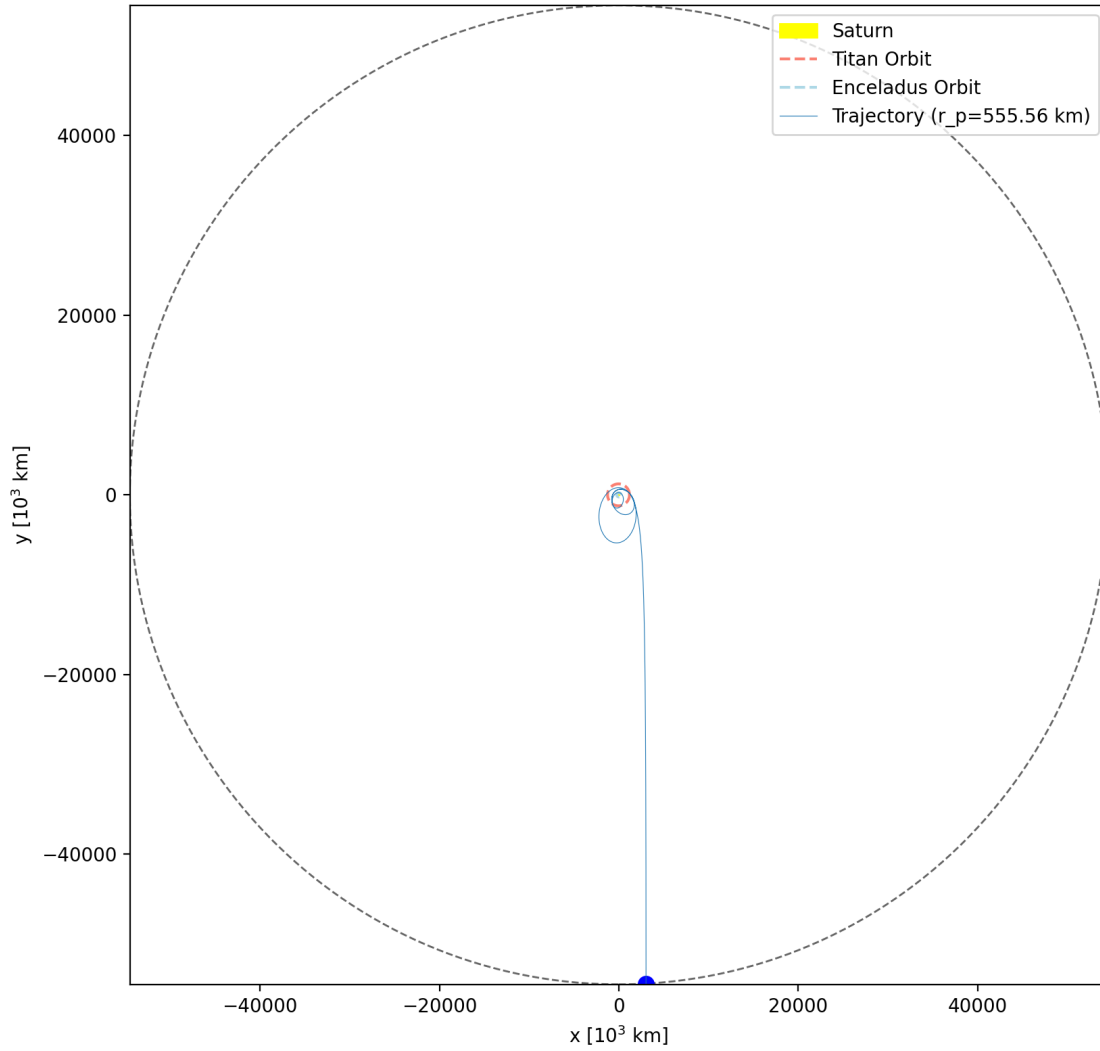


Figure A.2: Full Trajectory Plot



### A.3 Deviation in Aerothermodynamics

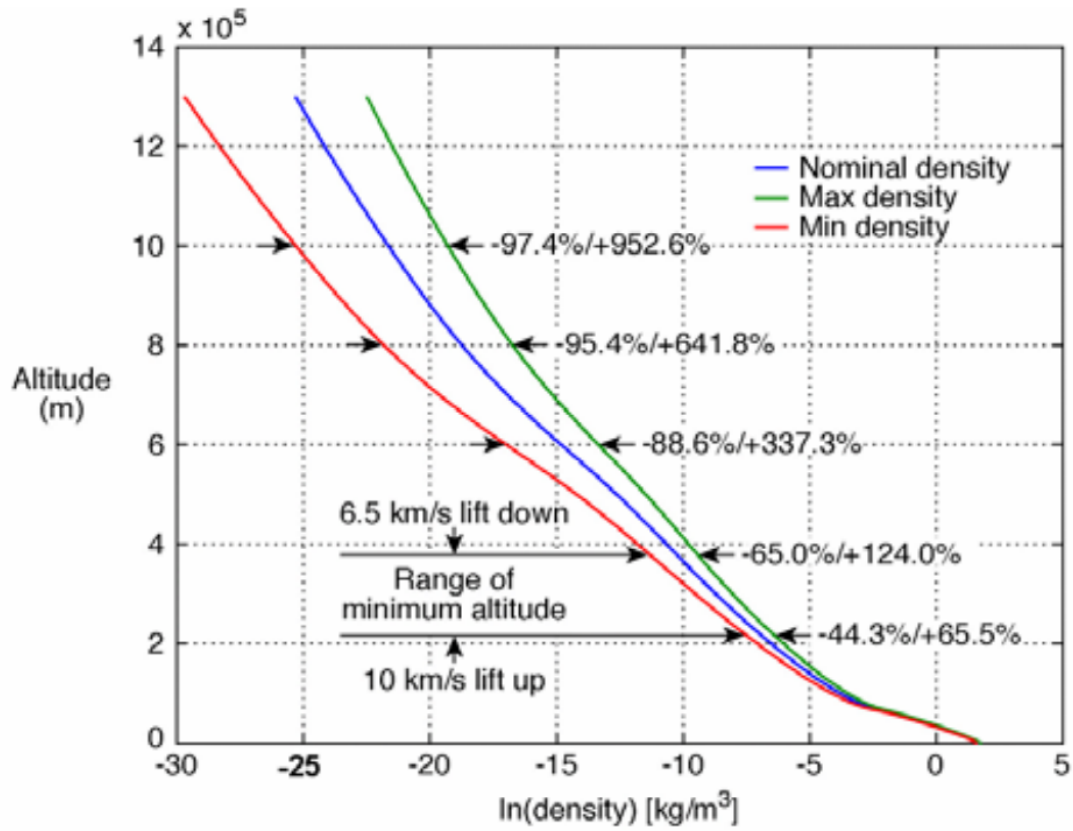


Figure A.3: Atmosphere model according to NASA aerothermodynamics study (Laub, 2012)

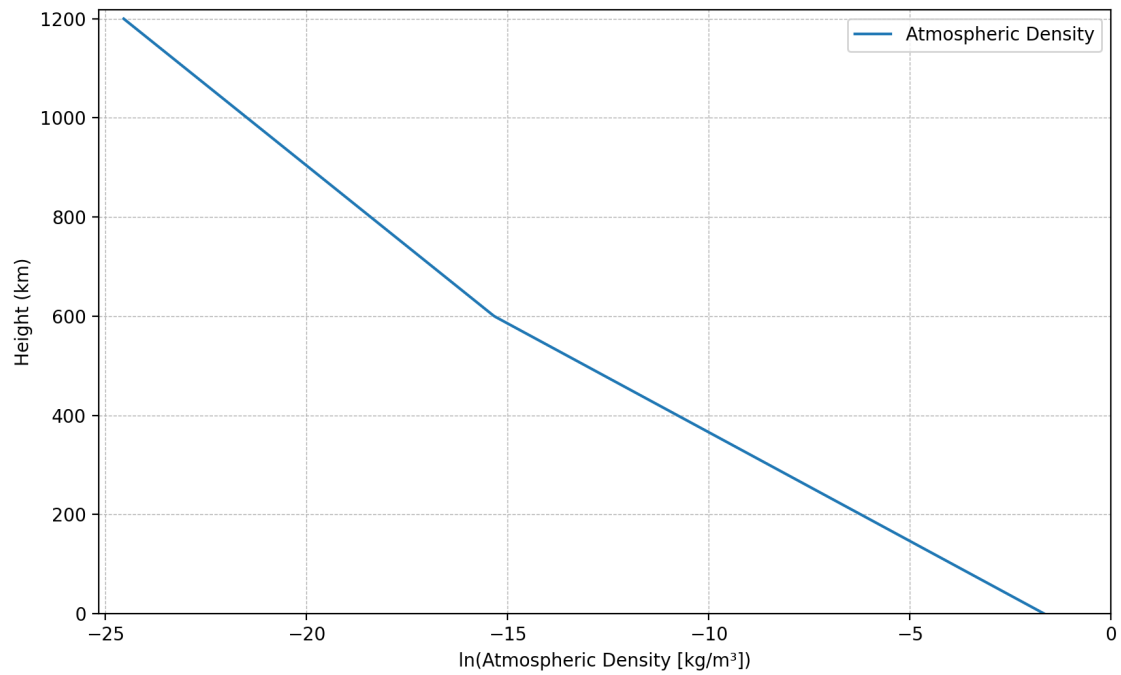


Figure A.4: Atmosphere model from TAGAS

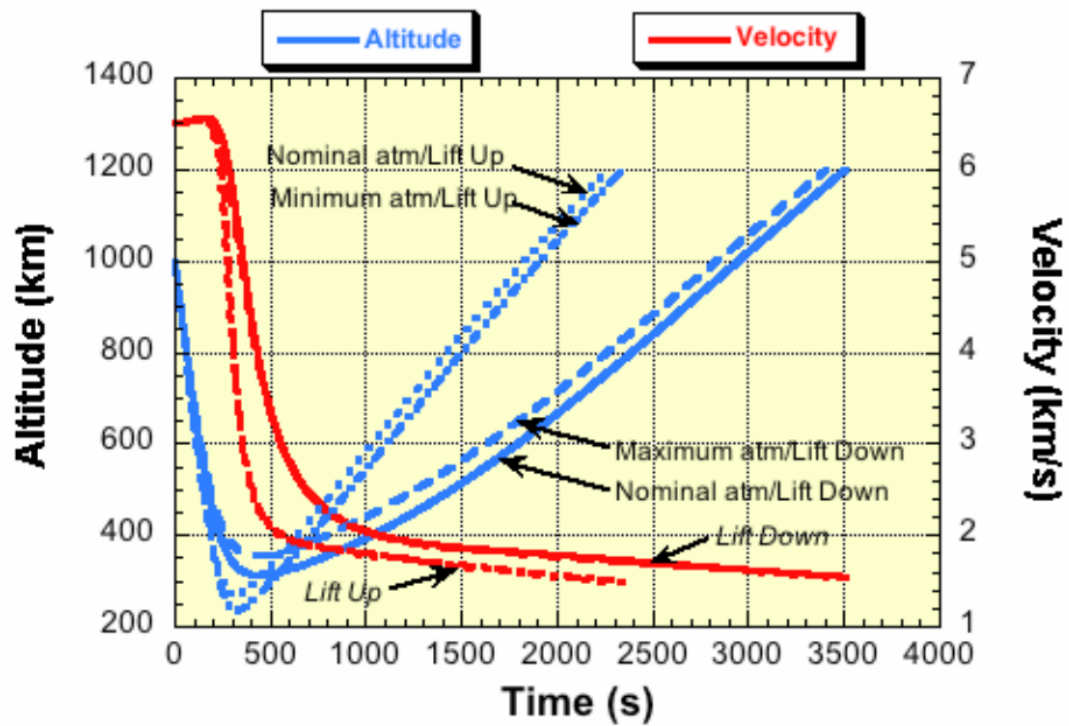


Figure A.5: Altitude & velocity during aerocapture according to NASA aerothermodynamics study (Laub, 2012))

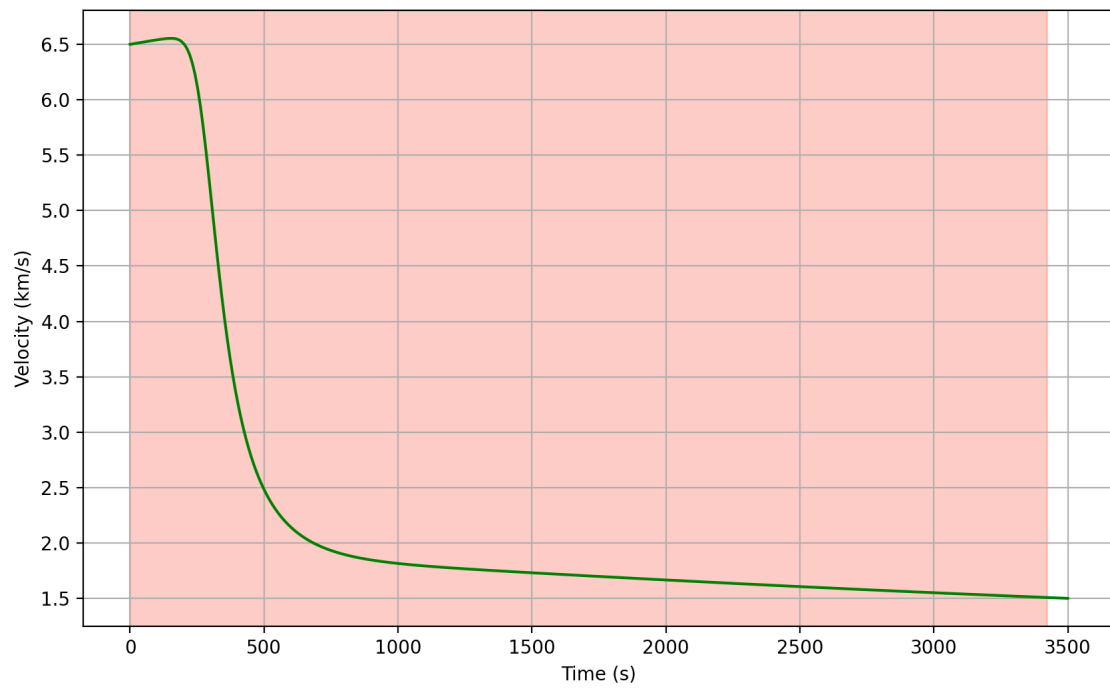


Figure A.6: Velocity during aerocapture from TAGAS

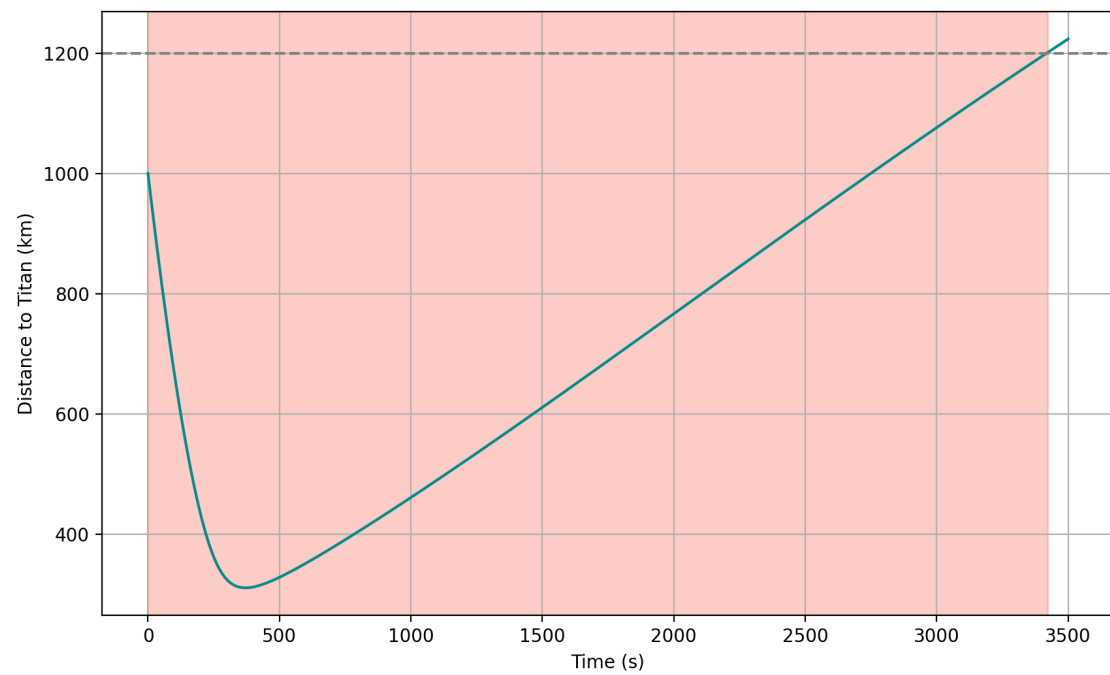


Figure A.7: Altitude during aerocapture from TAGAS

## B Reference Frames

An inertial reference frame is a frame that is not accelerated and does not rotate. In this frame Newton's laws are valid.

### B.1 Inertial Cartesian Reference Frame $\mathcal{I}$

The 3-dimensional  $\mathcal{I}$ -frame contains the three unit vectors  $\hat{x}$  in x-direction and  $\hat{y}$  in y-direction and  $\hat{z}$  in z-direction with the origin  $\mathcal{O}$ . It is an inertial orthogonal right-handed cartesian coordinate frame. Any position in this frame can be described with the position vector  $\mathbf{r}$  consisting of the components  $\{x,y,z\}$  as shown in Figure B.1 (Dachwald, 2024a). Since this thesis is working in the 2-dimensional space the  $\hat{z}$  vector can be neglected and  $\{x,y\}$  are sufficient to describe any position in this frame.

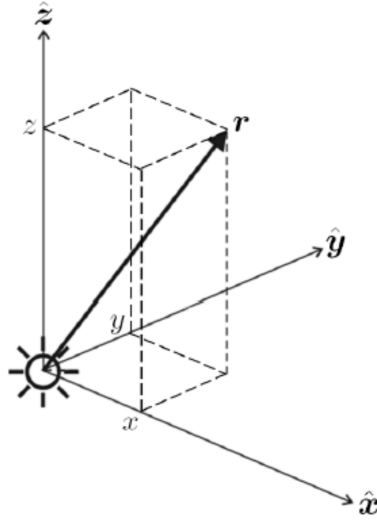


Figure B.1: Cartesian reference frame (Dachwald, 2024a)

## B.2 Orbit Reference Frame $\mathcal{P}$

The 2-dimensional  $\mathcal{P}$ -frame contains the radial unit vector  $\hat{\mathbf{r}}$  and the transversal unit vector  $\hat{\mathbf{t}}$ . The radial vector  $\hat{\mathbf{r}}$  points from the central body to the orbiting body. The transversal unit vector is defined so that  $\hat{\mathbf{r}} \times \hat{\mathbf{t}}$  creates a third vector, that is perpendicular to the orbit plane Figure B.2 (Dachwald, 2023).

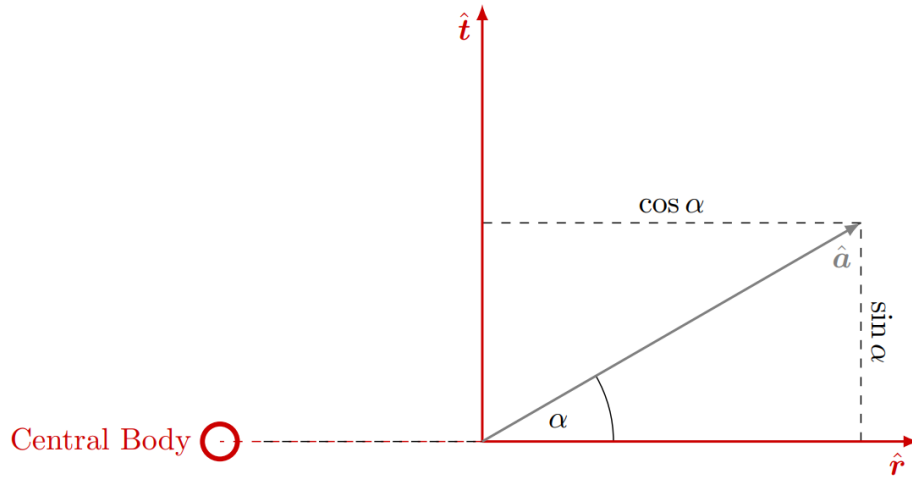


Figure B.2: Orbit reference frame (Dachwald, 2023)

It is important to note that the  $\mathcal{P}$ -frame is not an inertial frame, because it follows the rotation of the orbiting body. Using the angle  $\varphi$  as the angle of rotation of the  $\mathcal{P}$ -frame with respect to the  $\mathcal{I}$ -frame, the  $\mathcal{P}$ -frame unit vectors, expressed in  $\mathcal{I}$ -frame components, are (Dachwald, 2023):

$$\hat{\mathbf{r}} = \cos \varphi \hat{\mathbf{x}} + \sin \varphi \hat{\mathbf{y}} \quad (\text{B.1})$$

$$\hat{\mathbf{t}} = -\sin \varphi \hat{\mathbf{x}} + \cos \varphi \hat{\mathbf{y}} \quad (\text{B.2})$$

# C Python Code

The entire python code is available for download at: <https://github.com/CedricKalscheuer/Aerobraking>.

## C.1 Launch Delay Iteration Process

```
561 #region Launch Time
562 def calculate_launch_delay_iterative(initial_launch_time, r_p, v_inf, radial_tolerance_km=5, initial_delay_time=None):
563
564     rp_matching_tol = 4
565     if initial_delay_time is None: # Check if a delay time for a similar configuration is in the database
566         for cfg in r_p_configurations.get(v_inf, []):
567             if abs(cfg["first_crossing_rp"] - r_p) <= rp_matching_tol:
568                 initial_delay_time = cfg.get("initial_delay", 0)
569                 break
570         else:
571             # Fallback if none matched
572             initial_delay_time = 0
573     delay_time = initial_delay_time
574
575     iteration = 1
576     max_iterations = 21
577
578     while iteration < max_iterations:
579         # "Stop after first crossing" only
580         _, _, trajectory, _, _ = simulate_trajectory(v_inf, r_p, short_run=True, launch_time=delay_time,
581             critical_distance=critical_distance_titan_sim, stop_after_first_crossing=True, radial_tolerance_km=radial_tolerance_km)
582
583         crossings_in_iteration = []
584         crossing_found = False
585         within_tolerance = False
586
587         for i in range(1, len(trajectory.y[0, :])):
588             # === Position Spacecraft/Titan ===
589             x_titan, y_titan, v_titan = get_titan_position(trajectory.t[i]) # Titan's position
590             r_spacecraft = trajectory.y[0, i] # Spacecraft radial position
591             phi_spacecraft = trajectory.y[1, i] # Spacecraft angular position
592             x_spacecraft = r_spacecraft * np.cos(phi_spacecraft) # Spacecraft's position in Cartesian coordinates
593             y_spacecraft = r_spacecraft * np.sin(phi_spacecraft)
594
595             # === Distance Calculations ===
596             distance_to_titan = np.sqrt((x_spacecraft - x_titan)**2 + (y_spacecraft - y_titan)**2) # Distance between spacecraft and Titan
597             distance_to_titan_surface = distance_to_titan - R_titan # Altitude above Titan's surface
598             r_titan_at_phi = np.sqrt(x_titan**2 + y_titan**2) # Titan's radial distance from Saturn
599             radial_difference = abs(r_spacecraft - r_titan_at_phi) # Distance between spacecraft and Titans orbit
600
601             # Check if the spacecraft is within radial tolerance of Titan's orbit
602             if radial_difference <= radial_tolerance_km:
603                 if not within_tolerance: #Ensure a crossing only gets checked once until out of tolerance region again
604                     within_tolerance = True
605                     crossing_found = True
606                     time_at_orbit = trajectory.t[i]
607                     phi_spacecraft_at_orbit = phi_spacecraft
608                     x_titan_at_orbit = x_titan
609                     y_titan_at_orbit = y_titan
610                     v_titan_at_orbit = v_titan
611                     distance_to_titan_surface_at_orbit = distance_to_titan_surface
612                     crossings_in_iteration.append((i, time_at_orbit, phi_spacecraft_at_orbit, x_titan_at_orbit, y_titan_at_orbit, v_titan_at_orbit, distance_to_titan_surface_at_orbit))
```

```

613         else:
614             if within_tolerance:
615                 within_tolerance = False
616
617     if not crossing_found:
618         print(f"WARNING: The spacecraft did not cross Titan's orbit during iteration {iteration}.")
619         return delay_time, None, None, None, None
620
621     # Sort the crossings by time to ensure the first crossing is earliest
622     crossings_in_iteration.sort(key=lambda x: x[1]) # x[1] is time_at_orbit
623
624     # Print angular difference and distance to Titan for all crossings
625     for idx, crossing in enumerate(crossings_in_iteration):
626         ci, ctime_at_orbit, cphi_spacecraft_at_orbit, cx_titan_at_orbit, \
627         cy_titan_at_orbit, cv_titan_at_orbit, cdistance_to_titan_surface_at_orbit = crossing
628
629         cphi_titan_at_orbit = np.arctan2(cy_titan_at_orbit, cx_titan_at_orbit)
630         delta_phi = (cphi_spacecraft_at_orbit - cphi_titan_at_orbit + np.pi) % (2 * np.pi) - np.pi # Normalize to [-pi, pi]
631         delta_phi_deg = np.degrees(delta_phi) # Convert delta_phi to degrees
632
633     # Use the first crossing for delay time adjustment
634     crossing_for_adjustment = crossings_in_iteration[0]
635     i, time_at_orbit, phi_spacecraft_at_orbit, x_titan_at_orbit, \
636     y_titan_at_orbit, v_titan_at_orbit, distance_to_titan_surface_at_orbit = crossing_for_adjustment
637
638     phi_titan_at_orbit = np.arctan2(y_titan_at_orbit, x_titan_at_orbit)
639     delta_phi = (phi_spacecraft_at_orbit - phi_titan_at_orbit + np.pi) % (2 * np.pi) - np.pi # Normalize to [-pi, pi]
640     delta_phi_deg = np.degrees(delta_phi) # Convert to degrees
641
642     # Check if spacecraft is at the desired distance at the first crossing
643     distance_error = distance_to_titan_surface_at_orbit + R_titan - r_p
644     if abs(distance_error) <= tolerance:
645         print(f" [[v_inf:.2f] km/s] OPTIMAL DELAY: {delay_time / 86400:.12f} days")
646         return delay_time, None, None, None, None
647
648     # Adjust delay time based on the angular difference
649     if abs(delta_phi_deg) > 1:
650         r_current = np.sqrt(x_titan_at_orbit**2 + y_titan_at_orbit**2)
651         omega_avg = v_titan_at_orbit / r_current
652         delay_adjustment = delta_phi / omega_avg * 0.800
653         delay_time += delay_adjustment
654         print(f" [[v_inf:.2f] km/s] Delay-Iteration {iteration} for r_p = {r_p - R_titan:.6f} km: Spacecraft is {delta_phi_deg:.2f} degrees away from crossing {delay_time/86400:.14f} days")
655         #print(f" Adjusting delay by {delay_adjustment / 3600:.2f} hours to {delay_time/86400:.10f} days.")
656     else: # Fine adjustments for small angular differences
657         if delta_phi_deg > 0:
658             if abs(distance_error) >= 200:
659                 delay_adjustment = abs(distance_error) * 0.1 * (1/v_inf)**0.03
660             else:
661                 if abs(distance_error) < 0.0000002:
662                     delay_adjustment = abs(distance_error) * 0.08 * 0.020
663                 else:
664                     if v_inf <= 0.50:
665                         delay_adjustment = abs(distance_error) * 0.18
666                     elif v_inf <= 0.75:
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709

```

```

        delay_adjustment = abs(distance_error) * 0.184/1.5
        elif v_inf <= 1.0:
            delay_adjustment = abs(distance_error) * 0.183
        elif v_inf <= 2.0:
            delay_adjustment = abs(distance_error) * 0.133
        elif v_inf <= 2.5:
            delay_adjustment = abs(distance_error) * 0.11
        elif v_inf <= 3.0:
            delay_adjustment = abs(distance_error) * 0.095*1.7
        elif v_inf <= 3.5:
            delay_adjustment = abs(distance_error) * 0.07*1.3
        elif v_inf <= 4.0:
            delay_adjustment = abs(distance_error) * 0.07*0.8
        elif v_inf <= 5.0:
            delay_adjustment = abs(distance_error) * 0.07*0.7
        elif v_inf <= 6.0:
            delay_adjustment = abs(distance_error) * 0.09/5
        else:
            delay_adjustment = abs(distance_error) * 0.177 * v_inf**0.0213
    else:
        if abs(distance_error) >= 10000:
            delay_adjustment = abs(distance_error) * 0.2222222
        elif abs(distance_error) >= R_titan and abs(distance_error) < 10000:
            delay_adjustment = abs(distance_error) * 0.33333333
        else:
            delay_adjustment = abs(distance_error + R_titan) / 2.7
    # Adjust delay based on angular difference and distance to Titan
    if delta_phi_deg > 0: # Spacecraft is ahead of Titan
        if distance_to_titan_surface_at_orbit + R_titan < r_p:
            print(f" [[v_inf:.2f] km/s] Delay-Iteration {iteration} for r_p = {r_p - R_titan:.6f} km: Spacecraft is {abs(distance_error):.10f} km too close to Titan's surface. {delay_time/86400:.14f} days")
            delay_time -= delay_adjustment
        else:
            print(f" [[v_inf:.2f] km/s] Delay-Iteration {iteration} for r_p = {r_p - R_titan:.6f} km: Spacecraft is {abs(distance_error):.10f} km too far ahead of Titan. {delay_time/86400:.14f} days")
            delay_time += delay_adjustment
    else: # Spacecraft is behind Titan
        print(f" [[v_inf:.2f] km/s] Delay-Iteration {iteration} for r_p = {r_p - R_titan:.6f} km: Spacecraft is behind Titan ({distance_to_titan_surface_at_orbit:.8f} km), {delay_time/86400:.14f} days")
        delay_time += delay_adjustment
    iteration += 1
    # If we reach here, we have either converged or hit the maximum iterations
    print(f"Final launch delay for [v_inf = {v_inf:.2f} km/s] r_p = {r_p - R_titan:.4f} km: {delay_time / 86400:.14f} days")
#endregion

```

## C.2 Adjust Aerobrake Altitude Process

```

712 #region Adjust Aerobrake Altitude
713 def adjust_r_p_to_match_crossing(v_inf, target_crossing_number, target_crossing_distance, r_p_initial, tolerance_angle, tolerance_distance):
714     current_r_p = r_p_initial
715     iteration = 1
716     radial_tolerance_km = 5
717     max_iterations = 21
718
719     if target_crossing_number == 3:
720         if v_inf < 1:
721             k_angle_base = 1.15 * v_inf ** 0.21 * 0.1
722             k_distance_base = 0.00004 * v_inf ** 0.1 * 0.5
723         elif v_inf < 2:
724             k_angle_base = 1.1*0.1
725             k_distance_base = 0.00007*0.1
726         elif v_inf < 2.5:
727             k_angle_base = 1.25*0.1
728             k_distance_base = 0.000055*0.2
729         elif v_inf < 3:
730             k_angle_base = 0.2*0.6
731             k_distance_base = 0.000003*2.5
732         elif v_inf <= 3.5:
733             k_angle_base = 0.3*0.2
734             k_distance_base = 0.0000045*0.6
735         elif v_inf <= 4.0:
736             k_angle_base = 0.3*0.04
737             k_distance_base = 0.0000045*0.4
738         else:
739             k_angle_base = 0.3*0.0005
740             k_distance_base = 0.0000004
741     elif target_crossing_number == 6: ...
742     elif target_crossing_number == 8: ...
743     else:
744         k_angle_base = 10 * (1 / v_inf) ** 0.7
745         k_distance_base = 0.00015 * (1 / v_inf) ** 1.4
746
747     while iteration < max_iterations:
748         delay_time, _, _, _ = calculate_launch_delay_iterative(0, current_r_p, v_inf)
749
750         _, _, trajectory, _, _ = simulate_trajectory(v_inf, current_r_p, short_run=True, launch_time=delay_time,
751             critical_distance=critical_distance_titan_sim, stop_after_first_crossing=False)
752
753         within_tolerance = False
754         crossings_found = []
755         last_periapsis_value = None
756
757         r = trajectory.y[0, :]
758         phi = trajectory.y[1, :]
759         rhor = trajectory.y[2, :]
760
761         for i in range(len(trajectory.t)):
762             # Detect periapsis by finding where rhor switches from negative to positive
763             if i > 0 and rhor[i - 1] < 0 and rhor[i] > 0:
764                 last_periapsis_value = r[i]
765
766

```



```

802 # Check for radial tolerance crossings
803 x_titan, y_titan, v_titan = get_titan_position(trajjectory.t[i])
804 r_spacecraft = r[i]
805 phi_spacecraft = phi[i]
806 x_spacecraft = r_spacecraft * np.cos(phi_spacecraft)
807 y_spacecraft = r_spacecraft * np.sin(phi_spacecraft)
808 distance_to_titan = np.sqrt((x_spacecraft - x_titan)**2 + (y_spacecraft - y_titan)**2)
809 r_titan_at_phi = np.sqrt(x_titan**2 + y_titan**2)
810 radial_difference = abs(r_spacecraft - r_titan_at_phi)
811
812 if radial_difference <= radial_tolerance_km:
813     if not within_tolerance:
814         within_tolerance = True
815         phi_titan = np.arctan2(y_titan, x_titan)
816         delta_phi = (phi_spacecraft - phi_titan + np.pi) % (2 * np.pi) - np.pi
817         delta_phi_deg = np.degrees(delta_phi)
818         crossings_found.append((trajjectory.t[i], distance_to_titan, delta_phi_deg))
819     else:
820         if within_tolerance:
821             within_tolerance = False
822
823 if len(crossings_found) < target_crossing_number:
824     print(f"[{v_inf:.2f} km/s] NOT ENOUGH CROSSINGS. FOUND: {len(crossings_found)}, REQUIRED: {target_crossing_number}")
825     break
826
827 _, crossing_distance, crossing_angle_deg = crossings_found[target_crossing_number - 1]
828
829 #Check angle and distance conditions
830 if abs(crossing_angle_deg) > tolerance_angle or crossing_angle_deg < 0:
831     # Adjust r_p based on angle
832     if crossing_angle_deg < 0 and abs(crossing_angle_deg) < 1:
833         delta_r_p_angle = 7 * k_angle_base * crossing_angle_deg
834     else:
835         delta_r_p_angle = k_angle_base * crossing_angle_deg
836     print(f"[{v_inf:.2f} km/s] Error: {crossing_angle_deg:.2f} deg | CROSSING #{target_crossing_number} | R_P {current_r_p - R_titan:.10f} -> {current_r_p + delta_r_p_angle - R_titan:.10f} km | DELAY: {delay_time/86400:.14f} days")
837     current_r_p += delta_r_p_angle
838 else:
839     # Angle is within tolerance, now adjust based on distance
840     distance_error = crossing_distance - target_crossing_distance
841     if abs(distance_error) <= tolerance_distance:
842         crossings_to_display = [1, 3, 6, 8]
843         crossing_distances_str = " | ".join(
844             f"[{i}]: {crossings_found[i][1] - R_titan:.10f} km"
845             for i in range(target_crossing_number) if i+1 in crossings_to_display
846         )
847         print(f"ITERATION DONE FOR [{v_inf:.2f} km/s] | {crossing_distances_str} | LAST PERIAPSIS DISTANCE TO ENCELADUS: {(last_periapsis_value-a_enceladus):.0f} km | DELAY: {delay_time/86400:.14f} days")
848         if target_crossing_number <= 3:
849             print(
850                 f'{"first_crossing_rp": R_titan + {crossings_found[0][1] - R_titan:.10f}, '
851                 f'{"second_crossing_rp": R_titan + {crossings_found[2][1] - R_titan:.10f}, '
852                 f'{"third_crossing_rp": R_titan + {crossings_found[5][1] - R_titan:.10f}, '
853                 f'{"initial_delay": 86400 * {delay_time/86400:.14f}}',
854             )
855         elif target_crossing_number <= 6:
856             print(
857                 f'{"first_crossing_rp": R_titan + {crossings_found[0][1] - R_titan:.10f}, '
858                 f'{"second_crossing_rp": R_titan + {crossings_found[2][1] - R_titan:.10f}, '
859                 f'{"third_crossing_rp": R_titan + {crossings_found[5][1] - R_titan:.10f}, '
860                 f'{"initial_delay": 86400 * {delay_time/86400:.14f}}',
861             )
862         else:
863             print(
864                 f'{"first_crossing_rp": R_titan + {crossings_found[0][1] - R_titan:.10f}, '
865                 f'{"second_crossing_rp": R_titan + {crossings_found[2][1] - R_titan:.10f}, '
866                 f'{"third_crossing_rp": R_titan + {crossings_found[5][1] - R_titan:.10f}, '
867                 f'{"fourth_crossing_rp": R_titan + {crossings_found[7][1] - R_titan:.10f}, '
868                 f'{"initial_delay": 86400 * {delay_time/86400:.14f}}',
869             )
870         return current_r_p, delay_time, trajectory
871     else:
872         if current_r_p - R_titan <= 600:
873             delta_r_p_distance = k_distance_base / 2.0 * distance_error
874         elif current_r_p - R_titan <= atmo_height:
875             delta_r_p_distance = k_distance_base / 1.5 * distance_error
876         else:
877             delta_r_p_distance = k_distance_base * distance_error
878         print(f"[{v_inf:.2f} km/s] Error: {distance_error:.2f} km | CROSSING #{target_crossing_number} at {crossing_distance-R_titan:.10f} km |")
879         current_r_p += delta_r_p_distance
880
881     iteration += 1
882
883 print(f"[{v_inf:.2f} km/s] Did not converge within {max_iterations-1} iterations for crossing #{target_crossing_number}.")
884 return current_r_p, delay_time, trajectory
885 #endregion

```

# Bibliography

Ahrens, C. (2021). Hiten Mission.

<https://ntrs.nasa.gov/citations/20210016225> (visited on 02/28/2025).

Barnett, A. (2025). Saturn Moons.

<https://science.nasa.gov/saturn/moons/> (visited on 02/28/2025).

Bate, R. R., D. D. Mueller, and J. E. White (1971). Fundamentals of Astrodynamics.

<https://archive.org/details/fundamentalsofas00bate/mode/2up>  
(visited on 03/05/2025).

Brown, C. (1998). Spacecraft Mission Design. Published by the American Institute of Aeronautics and Astronautics.

<https://arc.aiaa.org/doi/10.2514/5.9781600862434.0095.0132>  
(visited on 02/28/2025).

Curtis, H. (2014). Orbital Mechanics for Engineering Students. Published by Butterworth-Heinemann/Elsevier.

<https://www.sciencedirect.com/book/9780080977478> (visited on 02/28/2025).

Dachwald, B. (2023). 2D Simulator for Low-Thrust Spacecraft. FH Aachen.

Dachwald, B. (2024a). Spaceflight Mechanics Lecture Chapter 2 Circular Orbits. FH Aachen.

Dachwald, B. (2024b). Spaceflight Mechanics Lecture Chapter 9 Interplanetary Trajectories. FH Aachen.

ESA (2014). Venus Express.

[https://www.esa.int/Enabling\\_Support/Operations/Venus\\_Express](https://www.esa.int/Enabling_Support/Operations/Venus_Express)  
(visited on 02/28/2025).

ESA (2017). Hang 10 Over Mars.

<https://blogs.esa.int/rocketscience/2017/03/16/hanging-10-over-mars/> (visited on 01/22/2025).

ESA (2024). Saturn's Moon Enceladus Top Target for Esa.

[https://www.esa.int/Science\\_Exploration/Space\\_Science/Saturn\\_s\\_moon\\_Enceladus\\_top\\_target\\_for\\_ESA](https://www.esa.int/Science_Exploration/Space_Science/Saturn_s_moon_Enceladus_top_target_for_ESA) (visited on 01/22/2025).

Gasch, M., J. Monk, H. Alpert, and E. Venkataphy (2022). Heatshield for Extreme Entry Environment Technology (Heeet) and 3D Woven Tps Readiness for Outer Planet Probe Missions.

<https://ntrs.nasa.gov/citations/20220017030> (visited on 02/19/2025).

Hall, N. (2023). Drag Equation.

<https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/drag-equation-2/> (visited on 03/08/2025).

Haynes, W. M. (2017). Handbook of Chemistry and Physics 97th Edition. Published by CRC Press Taylor Francis Group.

[https://mathguy.us/BySubject/Chemistry/CRC\\_Handbook\\_of\\_Chemistry\\_and\\_Physics\\_97.pdf](https://mathguy.us/BySubject/Chemistry/CRC_Handbook_of_Chemistry_and_Physics_97.pdf) (visited on 03/05/2025).

Jet Propulsion Laboratory (1993). Magellan Descends Into Venus' Atmosphere.

<https://www.jpl.nasa.gov/news/magellan-descends-into-venus-atmosphere/> (visited on 01/22/2025).

Jet Propulsion Laboratory (2019). Discipline Science Results Titan. *part of Cassini Final Mission Report.*

[https://pds-atmospheres.nmsu.edu/data\\_and\\_services/atmospheres\\_data/Cassini/Cassini/Cassini\\_Final\\_RPT/Cassssini%20Final%20Rpt%202019\\_Vol%201%20-%20Titansdpdf%20%5Bpreview%5D.pdf](https://pds-atmospheres.nmsu.edu/data_and_services/atmospheres_data/Cassini/Cassini/Cassini_Final_RPT/Cassssini%20Final%20Rpt%202019_Vol%201%20-%20Titansdpdf%20%5Bpreview%5D.pdf) (visited on 01/22/2025).

Jet Propulsion Laboratory (2022). Horizons System.

<https://ssd.jpl.nasa.gov/horizons/app.html#/> (visited on 02/10/2025).

Justus, C., A. Duvall, and D. Johnson (2003). Engineering-Level Model Atmospheres for Titan and Neptune.

<https://ntrs.nasa.gov/citations/20030066242> (visited on 02/28/2025).

Kalscheuer, J. C. (2025). Atmosphärenbremsung bei Marsmissionen.

<https://github.com/CedricKalscheuer/Aerobraking/blob/Praxisprojekt/Projektbericht.pdf> (visited on 03/02/2025).

Langley Research Center (2006). Aerocapture Systems Analysis for a Titan Mission.

<https://ntrs.nasa.gov/citations/20060007561> (visited on 02/26/2025).

Laub, B. (2012). Thermal Protection Concepts and Issues for Aerocapture at Titan.

<https://arc.aiaa.org/doi/10.2514/6.2003-4954> (visited on 03/04/2025).

Lindsey, L. (2024). Aegaeon.

<https://science.nasa.gov/saturn/moons/aegaeon/> (visited on 02/28/2025).

Nixon, C., F. Kirchman, J. Esper, D. Folta, and A. Mashiku (2016). Aerocapture Design Study for a Titan Polar Orbiter.

<https://ntrs.nasa.gov/citations/20170006876> (visited on 02/26/2025).

Parvathi, S. and R. Ramanan (2016). Direct Transfer Trajectory Design Options for Interplanetary Orbiter Missions Using an Iterative Patched Conic Method.

[www.researchgate.net/publication/313160426](http://www.researchgate.net/publication/313160426) (visited on 02/28/2025).

Reckart, T. (n.d.).

Roy, A. (2005). Orbital Motion. Published by CRC Press Taylor Francis Group.

<https://www.perlego.com/book/1645788> (visited on 02/28/2025).

Seal, D. (2002). Saturn's Satellites and Ring Structures.

[https://www.esa.int/Science\\_Exploration/Space\\_Science/Cassini-Huygens/Saturn\\_s\\_moons](https://www.esa.int/Science_Exploration/Space_Science/Cassini-Huygens/Saturn_s_moons) (visited on 02/28/2025).

Shao, G., Y. Lua, D. A. Hanaorc, S. Cui, J. Jiaod, and X. Shen (2019). Improved Oxidation Resistance of High Emissivity Coatings on Fibrous Ceramic for Reusable Space Systems.

<https://www.sciencedirect.com/science/article/abs/pii/S0010938X1830605X>  
(visited on 02/19/2025).

Wang, X.-M., L.-S. Zhang, C. Yang, N. Liu, and W.-L. Cheng (2019). Estimation of Temperature-Dependent Thermal Conductivity and Specific Heat Capacity for Charring Ablators.

<https://www.sciencedirect.com/science/article/abs/pii/S0017931018327169>  
(visited on 02/19/2025).

Way, D., R. Powell, K. Edquist, J. Masciarelli, and B. Starr (2003). Aerocapture Simulation and Performance for the Titan Explorer Mission.

<https://arc.aiaa.org/doi/10.2514/6.2003-4951> (visited on 02/26/2025).

Winter, G. (2021). Distances From Saturn.

<https://www.skymarvels.com/infopages/saturninfo.htm> (visited on 02/28/2025).

Wolf, A. A. and J. C. Smith (1995). Design of the Cassini Tour Trajectory in the Saturnian System.

<https://www.sciencedirect.com/science/article/abs/pii/096706619500172Q>  
(visited on 03/08/2025).

Wright, M. and J. Dec (2012). Aerothermodynamic and Thermal Protection System Aspects of Entry System Design Course.

<https://tfaws.nasa.gov/TFAWS12/Proceedings/Aerothermodynamics%20Course.pdf> (visited on 03/05/2025).

## DECLARATION

I hereby affirm that I have written this thesis independently and have not used any sources other than those listed in the bibliography.

Passages that have been taken literally or analogously from published or not yet published sources are marked as such.

The pictures or illustrations in this work have been made by myself or are accompanied by a corresponding reference.

This thesis has not yet been submitted in the same or a similar form to any other examination office.

Aachen, March 2025

A handwritten signature in black ink, reading "J. Kalscheuer", with a horizontal line extending to the right.

---

Jason Cedric Kalscheuer